PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
Departamento de Ingeniería Eléctrica
IEE2913 — Diseño Eléctrico (Capstone)

Grupo 10

Informe de Avance 2 (25 %)

Proyecto: SupaClock: Wearable Biométrico Modular
Integrantes: Tomás Avendaño, Benjamín Sepúlveda, Pablo Uribe
Fecha: 4 de mayo de 2026

Índice

1. Introducción al estado actual del proyecto **5**
1.0.1. Resumen ejecutivo del avance. . . . 5

2. Diagrama de bloques de bajo nivel actualizado **5**
2.0.1. Cambios respecto del diagrama original. . . . 6
2.0.2. Especificaciones detalladas por bloque. . . . 7

3. Planificación actualizada **8**
3.1. Contraste avance esperado vs. avance logrado . . . 8
3.2. Carta Gantt corregida . . . 9
3.3. Distribución del trabajo en este hito . . . 10

4. Análisis de impacto y estándares **11**
4.1. Aplicaciones e impacto socioeconómico, ambiental y en salud . . . 11
4.1.1. Salud pública. . . . 11
4.1.2. Impacto socioeconómico y de equidad. . . . 11
4.1.3. Impacto ambiental. . . . 12
4.1.4. Aspectos éticos. . . . 12
4.2. Riesgo eléctrico — análisis ECMA-287 punto 3 . . . 12
4.2.1. Riesgos adicionales del LiPo. . . . 13
4.3. Estándares relacionados con el área del proyecto . . . 13
4.3.1. Bluetooth Core Specification 5.3 [1]. . . . 13
4.3.2. IEEE 11073-10406 (*Personal Health Devices — Basic ECG*) [2]. . . . 14
4.3.3. USB Type-C Cable and Connector Specification, Rev. 2.3 [3]. . . . 14
4.3.4. IEC 60601-1 (*Medical electrical equipment*) [4]. . . . 14
4.3.5. RoHS-2 (Directiva 2011/65/EU) [5] e IEC 62133-2 [6] . . . 14

5. Modelos y simulaciones **14**
5.1. Modelo de autonomía energética . . . 15
5.1.1. Disclaimer instrumental. . . . 15
5.1.2. Cota superior del consumo medio. . . . 15
5.2. Validación del algoritmo de pasos . . . 15
5.2.1. Resultado. . . . 16
5.2.2. Limitación detectada y migración planificada. . . . 16

5.3. Validación del algoritmo Pan-Tompkins . . . 16
5.3.1. Resultado. . . . 17
5.4. Modelo de concurrencia I2C bajo FreeRTOS . . . 18
5.4.1. Cálculo. . . . 18
5.4.2. Validación experimental. . . . 18
5.5. Modelo de duty cycle bajo PM dinámico . . . 18
5.5.1. Impacto de las mitigaciones. . . . 19
5.6. Análisis comparativo de topología algorítmica frente a la industria . . . 19

6. Diseño del set-up final de pruebas del producto integrado 20
6.1. Arquitectura mecánica del producto final . . . 20
6.1.1. Carga y alimentación. . . . 20
6.1.2. Batería. . . . 21
6.1.3. Ventana óptica del MAX30102. . . . 21
6.1.4. Acople térmico del MAX30205. . . . 21
6.1.5. Electrodos ECG. . . . 21
6.1.6. Carcasa. . . . 21
6.2. Plan de iteraciones de hardware . . . 21
6.3. Banco de pruebas mecánicas (caja + integración) . . . 22
6.4. Banco de pruebas eléctricas y funcionales (unidad cerrada) . . . 22
6.5. Pruebas de uso (*team testing*) . . . 23
6.6. Esquema visual del producto integrado . . . 23

7. Diseño de pruebas preliminar 25
7.1. Estrategia general . . . 25
7.2. Pruebas de integración . . . 26
7.3. Pruebas funcionales (sistema completo) . . . 27
7.4. Plan de validación de la PCB propia . . . 27

8. Implementación y resultados 27
8.1. Arquitectura de firmware concurrente (FreeRTOS) . . . 27
8.1.1. Descripción y funcionalidad. . . . 27
8.1.2. Cálculos / decisiones. . . . 28
8.1.3. Brown-out y secuencia de arranque. . . . 28
8.1.4. Resultados. . . . 29
8.2. Modos de energía y persistencia NVS . . . 29
8.2.1. Descripción. . . . 29
8.2.2. Persistencia NVS. . . . 29
8.2.3. Resultado. . . . 29
8.3. Power management dinámico (light sleep) . . . 30
8.3.1. Descripción. . . . 30
8.3.2. PM locks implementados. . . . 30
8.3.3. Resultado cuantitativo. . . . 30
8.4. Migración a Seeed XIAO ESP32-S3 . . . 30
8.4.1. Motivación técnica. . . . 30
8.4.2. Estado. . . . 31
8.5. Bloque de Sensores: BMI160 + algoritmo de pasos . . . 31
8.5.1. Descripción. . . . 31
8.5.2. Algoritmo de pasos (versión C3). . . . 31
8.5.3. Resultados. . . . 32
8.6. Bloque de Sensores: MAX30102 (PPG) . . . 32

8.6.1. Descripción. 32
8.6.2. Máquina de estados SPOT. 32
8.6.3. Motion gating. 33
8.6.4. Resultados. 33
8.7. Bloque de Sensores: AD8232 (ECG) con ADC continuo + DMA 33
8.7.1. Descripción. 33
8.7.2. Refactor DMA on-demand. 33
8.7.3. Cálculos. 33
8.7.4. Resultados. 34
8.8. Bloque de Sensores: MAX30205 (Temperatura) 34
8.8.1. Descripción. 34
8.8.2. Resultados. 34
8.9. Bloque de Energía: BMS + LDO + Fuel Gauge 34
8.9.1. Descripción. 34
8.9.2. Validación eléctrica. 34
8.9.3. Resultados. 34
8.10. Bloque de Comunicaciones: BLE NimBLE 35
8.10.1. Descripción. 35
8.10.2. Optimizaciones de consumo. 35
8.10.3. Resultados. 35
8.11. Bloque de Interfaz Local: GUI multi-pantalla LVGL 35
8.11.1. Descripción. 35
8.11.2. Menú principal (7 ítems). 36
8.11.3. Backlight PWM y auto-off. 36
8.11.4. Resultados. 36
8.12. Bloque de Procesamiento: Pan-Tompkins en NumPy (validado offline) 36
8.12.1. Descripción. 36
8.12.2. Modelo de costos (proyección hito 60 %). 37
8.12.3. Resultados. 37
8.13. Bloque de Hardware: Esquemático y prototipo PCB 37
8.13.1. Descripción. 37
8.13.2. Estado. 37
8.13.3. Evidencia visual. 37
8.14. Evaluación general y próximos pasos 38

A. Anexos: extractos de código **40**

Índice de figuras

1. Diagrama de bloques de bajo nivel actualizado del prototipo SupaClock. Cada bloque indica el porcentaje de implementación actual. Verde: funcional; amarillo: parcial; rojo punteado: pendiente o dependiente de la PCB SMD. 6
2. Carta Gantt actualizada estructurada en fases, con ajuste de pesos y estado de avance consolidado (45 %) al 04/05/2026. 10
3. Consumo medido en batería (Tektronix TX3) para distintos escenarios operativos post-optimización. El pico de 69 mA corresponde al peor caso: ECG + BLE streaming + pantalla encendida. El modo ECO con pantalla apagada alcanza 17 mA en estado activo. 15

4. Comparación de las dos variantes del algoritmo de pasos sobre ∼45 s de captura del BMI160 (*ground-truth* = 67 pasos). Panel superior: variante C3 (umbral midpoint en el dominio del tiempo, detectó 60). Panel inferior: variante S3 (FFT 128 puntos sobre ventanas de 2,56 s, detectó 65). Las marcas verticales indican los eventos de paso reportados por cada algoritmo. 16
5. Pipeline completo de procesamiento ECG: edge (ESP32) → cliente PC de testing → Firebase (planificado para el hito 60 %). En este avance la detección de R-peaks se ejecutó offline en Python+NumPy con `scripts/test_pt.py` contra los CSV capturados por `supaclock_monitor.py`. El despliegue como Cloud Function y el reemplazo del cliente PC por la app Flutter constituyen el siguiente hito. 17
6. Resultados del análisis Pan-Tompkins sobre datos reales del AD8232: señal ECG cruda, etapas de filtrado y detección de R-peaks. BPM calculado: 76; HRV (SDNN): 32,4 ms. . . 17
7. Fracción de tiempo del SoC en cada estado de potencia, antes y después de las optimizaciones. Antes: 0 % SLEEP, 92 % APB_MAX, 8 % CPU_MAX. Después: 75 % SLEEP, 15 % APB_MAX, 9 % CPU_MAX. 19
8. Distribución de CPU por tarea tras la optimización. El sistema pasa más del 95 % del tiempo en IDLE, lo que habilita el *light sleep* dinámico. El driver ST7789 reescrito redujo la carga de `gui_task` de 47 % a <1 %. 19
9. Bosquejo explosionado de la iteración v1 (pin sockets): carcasa PLA ∼50×40×13 mm, PCB única con módulos en socket (cara superior) y sensores MAX30102/MAX30205 en la cara inferior, batería LiPo reciclada, placa de aluminio para acople térmico, pernos M3 304 como electrodos ECG con contacto vía pogo pins, y lugs de 20 mm para correa. 24
10. Set-up final de validación. El DUT (*SupaClock*) se evalúa contra instrumentos de referencia (oxímetro, termómetro) y se registran las salidas BLE en CSV vía `supaclock_monitor.py` (PC) o la app Flutter (Android). 25
11. Arquitectura FreeRTOS implementada en `src/tests/test_general.c`. Siete tareas con prioridades distintas, dos mutex y *queue* implícita por el host de NimBLE. El bus I2C es serializado por el mutex `xSensorDataMutex`. 28
12. Dos pipelines de procesamiento del BMI160: arriba el contador de pasos (implementado), abajo la futura clasificación HAR vía CNN 1D (en desarrollo). 32
13. Vista frontal del prototipo SupaClock en protoboard: ESP32-C3 SuperMini, display ST7789 1.69”, módulo TP4056 y MAX17048 STEMMA. 38
14. Vista trasera del prototipo: sensores MAX30102 (PPG) y MAX30205 (temperatura) en contacto con la piel, módulo AD8232 con electrodos laterales. 38

Índice de cuadros

1. Especificaciones de bajo nivel por bloque y estado de implementación. 8
2. Contraste entre actividades planificadas preliminarmente y el avance logrado al 4 de mayo de 2026. 9
3. Distribución de trabajo y contribuciones principales de la iteración actual. 11
4. Análisis de riesgos eléctricos según ECMA-287, punto 3 [7]. 13
5. Plan de iteraciones de hardware del producto integrado. 22
6. Mapa entre bloques, entornos PlatformIO y criterios de aceptación de las pruebas unitarias. 26
7. Tareas FreeRTOS implementadas, prioridades y periodicidades. 28
8. Cadencias parametrizadas por modo de energía (extracto de `power_profiles[]`). 29

1. Introducción al estado actual del proyecto

El siguiente informe detalla el estado actual del desarrollo del proyecto *SupaClock*, dando continuidad al diseño establecido en el informe de diseño preliminar entregado el 7 de abril de 2026. La motivación, los requerimientos técnicos y el diagrama de bloques de alto nivel descritos en aquel documento se mantienen vigentes, por lo que en este informe **no se reproducen**, sino que se referencian. El presente documento se estructura en torno a cinco ejes fundamentales: diagrama de bloques de bajo nivel *actualizado*, planificación contrastada con el estado real del proyecto, análisis de impacto socioeconómico y de estándares aplicables, modelos y simulaciones que respaldan las decisiones de diseño, set-up final de pruebas y un diseño de pruebas preliminar (sección que se incorpora en este reporte para complementar la planificación original). Adicionalmente, en la **sección de Implementación y resultados** se documentan todos los bloques que se logró integrar en la etapa actual de desarrollo.

1.0.1. Resumen ejecutivo del avance.

Actualmente el sistema cuenta con un **firmware funcional integrado** que ejecuta **siete tareas concurrentes** bajo FreeRTOS (IMU + pasos, PPG con máquina de estados SPOT, Temperatura/Batería, ECG por DMA on-demand, GUI LVGL de 7 pantallas, flush BLE agregado y monitor de rendimiento), un **stack BLE NimBLE** operativo con **cuatro características GATT** (incluyendo telemetría TLV agregada y canal de comandos), una implementación del algoritmo **Pan-Tompkins** en Python+NumPy **validada offline** contra trazas reales de ECG (la integración con Firebase Cloud Functions se reservó para el hito 60 %), y un **esquemático en KiCad** para el prototipo SuperMini. Adicionalmente, en esta iteración se implementaron **modos de energía persistentes** (SPORT/NORMAL/SAVER con 8 cadencias parametrizadas y persistencia NVS), un sistema de **power management dinámico** con DFS y *light sleep* que alcanza un 75 % de tiempo en estado SLEEP, y se encargó la **migración al Seeed XIAO ESP32-S3** para resolver limitaciones de memoria y potencia del C3. Los componentes de Machine Learning (CNN 1D para HAR) y la PCB SMD propia están en fases tempranas y se concentran en los hitos 60 % y 90 %. Se inició el desarrollo de la **aplicación móvil en Flutter** que actuará como *gateway* entre el dispositivo y el backend; para la fase de diseño actual, las trazas BLE se validan con el cliente de escritorio `supaclock_monitor.py` (Python + `bleak`), que grafica biometría en tiempo real y exporta CSV. La integración *end-to-end* con Firebase (autenticación, reglas Firestore, despliegue de Cloud Functions) está planificada para el hito 60 %.

2. Diagrama de bloques de bajo nivel actualizado

El diagrama de alto nivel presentado en la *Fig. 4* de la entrega anterior se mantiene **sin modificaciones estructurales**: los siete dominios funcionales (interacción, energía, sensores, MCU, interfaz local, gateway y backend) y sus interconexiones siguen siendo válidos. Lo que sí se actualizó es el **nivel de detalle de bajo nivel**, dado que ahora cada bloque tiene un grado de implementación cuantificable sobre placa de desarrollo. La Fig. 1 muestra el estado vigente, codificando con color el porcentaje de integración y diferenciando con un borde discontinuo aquellos bloques que aún no están implementados o cuyo paso a la PCB propia está pendiente.

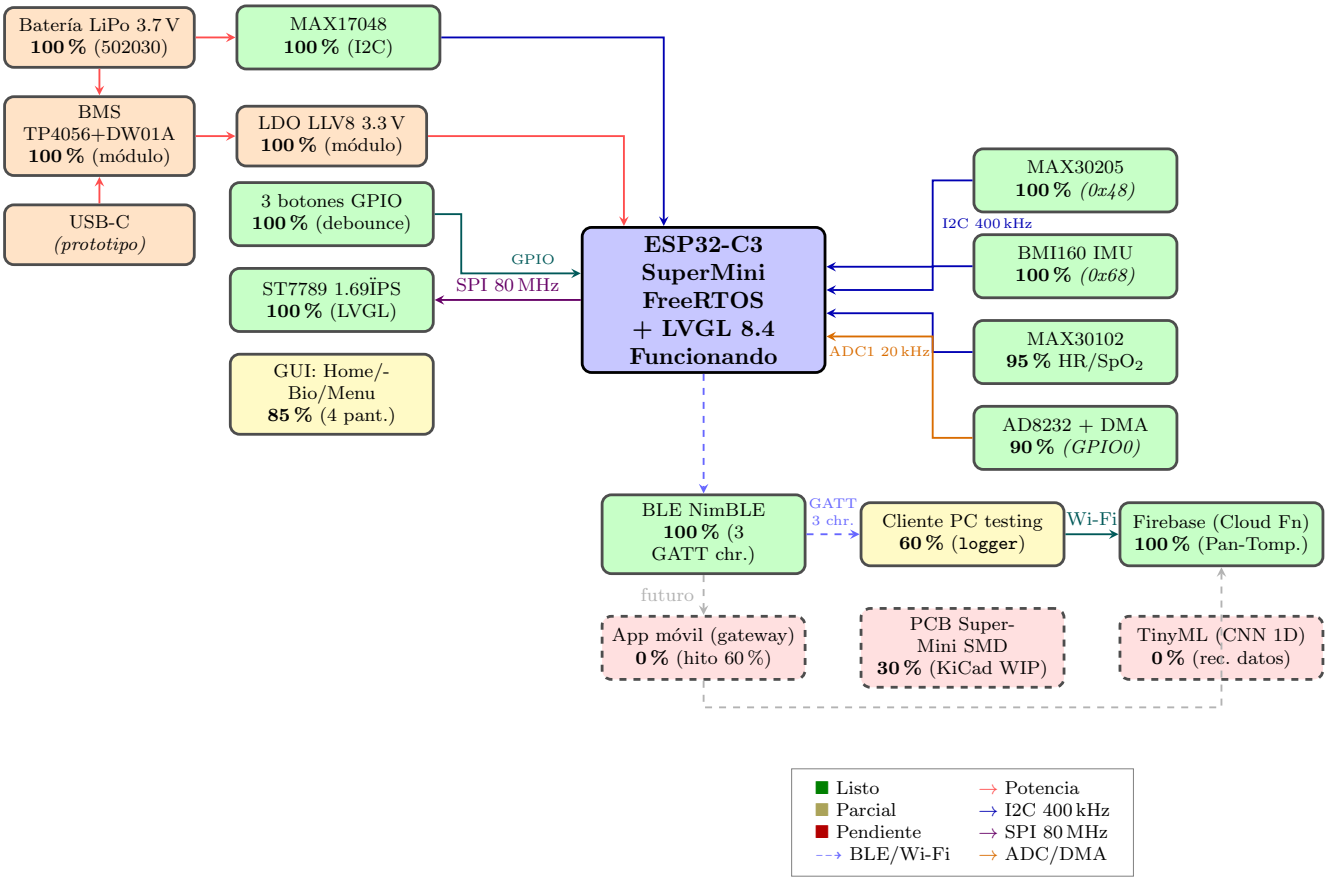


Figura 1: Diagrama de bloques de bajo nivel actualizado del prototipo SupaClock. Cada bloque indica el porcentaje de implementación actual. Verde: funcional; amarillo: parcial; rojo punteado: pendiente o dependiente de la PCB SMD.

2.0.1. Cambios respecto del diagrama original.

La estructura general del bus es la misma; las modificaciones son:

- **Bus I2C**: confirmado a 400 kHz con cuatro esclavos físicos (MAX30102 `0x57`, MAX30205 `0x48`, BMI160 `0x68`, MAX17048 `0x36`), accedido bajo el mutex `xSensorDataMutex` (FreeRTOS) para serializar las cuatro tareas concurrentes que comparten el bus.
- **ADC continuo (ECG)**: el AD8232 ya *no* se concibe como un canal ADC *single-shot*; la implementación actual usa el periférico `adc_continuous` del ESP-IDF a 20 kHz con DMA, decimando por 40 hacia 500 Hz para BLE. Esto se justifica en la Fig. 11.
- **Botones**: pasaron de tres a **dos botones** (`NEXT` GPIO10 y `SELECT` GPIO1) *con short-press y long-press* gestionados por `gpio_buttons` (debounce SW por `esp_timer`); el pin de despertar de *deep sleep* se canaliza ahora por `BTN_SELECT`.
- **LVGL**: se confirmó el buffer parcial de 14.4 KB (240×30 px × 2 B); el GUI quedó dividido en **siete pantallas** (5 ciclables: Home, Bio, HR_Spot, ECG, Menú; 2 sub-pantallas: Modo Energía, Configuración Auto-off) con navegación por estado.
- **Backend Firebase**: el algoritmo Pan-Tompkins (`firebase/functions/main.py`) fue **escrito y validado offline** en este avance, ejecutándolo con `scripts/test_pt.py` contra los CSV crudos del AD8232. El despliegue como Cloud Function (`firestore_fn.on_document_created`), la autentica-

ción Google y las reglas de Firestore se concentrarán en el hito 60 %, en paralelo con la primera versión del *gateway* móvil en Flutter.

- **Migración a ESP32-S3**: se encargó el módulo *Seeed XIAO ESP32-S3* (en tránsito) para resolver las limitaciones de potencia (brown-outs del LDO actual) y de memoria (techo SRAM de 320 KB insuficiente para TinyML). La decisión se justifica cuantitativamente en la sección 5.
- **TinyML**: aún *no* se entrenó el modelo CNN 1D (depende de la recolección de datos reales del BMI160 sobre los integrantes); por eso se mantiene como bloque pendiente. El contador de pasos *algorítmico* (umbral adaptativo) sí se implementó.
- **PCB SMD propia**: se completó la primera versión del esquemático en KiCad incluyendo la sección de potencia, los footprints importados y los buses I2C/SPI. Sin embargo, esta versión **no será manufacturada** dado que la migración al ESP32-S3 requiere un rediseño del módulo MCU; ya se trabaja en la versión actualizada para dejar definitivamente la protoboard.

2.0.2. Especificaciones detalladas por bloque.

La Tabla 1 consolida las especificaciones cuantitativas que permiten cerrar el diagrama. La columna "Estado" usa la misma codificación cromática que la Fig. 1.

Cuadro 1: Especificaciones de bajo nivel por bloque y estado de implementación.

Bloque	**Interfaz**	**Frecuencia / tasa**	**Especificación clave**	**Estado**
AD8232 (ECG)	ADC1 / DMA	20 kHz → 500 Hz (down-sample 40×)	GPIO0 ADC, ganancia analógica fija 1100, f_c LP ∼40 Hz	90 %
BMI160 (IMU)	I2C 400 kHz `0x68`	ODR 50 Hz	accel ±2 g, gyro ±250 °/s, FIFO HW, INT1 step counter	100 %
MAX30102 (PPG)	I2C 400 kHz `0x57`	100 sps efect. ($N_{avg} = 4$)	RED+IR pulse 411 μs, FIFO 32, polling 100 ms, flush startup	95 %
MAX30205 (Temp)	I2C 400 kHz `0x48`	1 Hz polling	16-bit, 0.00390625 °C/LSB, ±0,1 °C en rango clínico	100 %
MAX17048 (Fuel)	I2C 400 kHz `0x36`	1 Hz polling	ModelGauge SOC %, V_{cell} mV, sin shunt externo	100 %
ST7789 (Display)	SPI 80 MHz, 4-wire	30 FPS LVGL	240×280 RGB565, DMA `SPI_DMA_CH_AUTO`, offset Y=20	100 %
Botones GPIO	GPIO IN pull-up	polling 30 Hz desde gui_task	2 botones (NEXT, SELECT), short/long press, debounce 30 ms, wake-up deep sleep	100 %
BMS TP4056 + DW01A	USB-C IN	–	carga 1 A ajustable, OVP/OCP/UVP integrado	100 %
LDO ME6211	3.3 V OUT	–	dropout ∼100 mV @ 100 mA, I_Q 55 μA	100 %
BLE NimBLE	2.4 GHz BLE 5	MTU 247, conn. interval 15 ms	4 chr.: 0xFF01 IMU, 0xFF02 TLV agregado, 0xFF03 ECG, 0xFF04 CMD	100 %
GUI LVGL	SPI	30 FPS	7 pantallas (5 ciclables + 2 sub), 7 ítems menú, backlight PWM auto-off	100 %
Power Management	esp_pm + locks	DFS 10–160 MHz	Light sleep dinámico, 3 PM locks (ECG, ADC, NimBLE)	100 %
Power Modes	NVS + `power_get_profile()`	–	3 perfiles (SPORT/NORMAL/SAVER), 8 cadencias parametrizadas, persistencia NVS	100 %
Pan-Tompkins (algoritmo)	Python + NumPy	offline	5 etapas (BPF 5–15 Hz, deriv., cuad., MWI, R-peaks adaptativos), BPM y HRV (SDNN)	70 %
Cloud Functions	Firestore trigger	–	Despliegue, autenticación y reglas: hito 60 %	0 %
TinyML CNN 1D	TFLite Micro	0.5 Hz inferencia	4 clases HAR, <30 KB flash, ≤10 KB SRAM	0 %
PCB SuperMini propia	4 capas JLCPCB	–	Esquemático v1 completo; rediseño en curso para ESP32-S3	40 %

3. Planificación actualizada

3.1. Contraste avance esperado vs. avance logrado

La Tabla 2 contrasta las tareas de la planificación preliminar con su estado actual. Las filas en verde indican tareas completadas; las amarillas, tareas en curso; las rojas, tareas *adelantadas* respecto del cronograma original o tareas nuevas que se incorporaron al descubrir necesidades de implementación que no estaban en la planificación inicial.

Cuadro 2: Contraste entre actividades planificadas preliminarmente y el avance logrado al 4 de mayo de 2026.

Tarea planificada	Estado	Observaciones
Implementación bloque de energía (BMS + LDO)	**Completado**	Módulo TP4056 + DW01A montado y validado a 4.2 V; LDO ME6211 entrega 3.30 V estables. Driver `max17048` entrega SOC y V_{cell} con 1 Hz polling.
Integración inicial de sensores en bus I2C compartido	**Completado**	Cuatro esclavos en el bus a 400 kHz, mutex FreeRTOS protege accesos concurrentes desde 4 tareas.
Pruebas preliminares de adquisición de datos	**Completado**	Cada `env:test_*` de PlatformIO valida un sensor de forma aislada; existen 9 entornos de prueba unitarios.
Primera integración de sensores con el MCU	**Completado**	`env:main_app/test_general` integra simultáneamente IMU, PPG, Temp, ECG, batería y GUI con BLE activo.
Validación básica de algoritmos	**Completado**	Contador de pasos por umbral adaptativo en C (validado contra ground-truth manual) y algoritmo Pan-Tompkins en Python+NumPy (validado offline contra CSV crudo del AD8232). Ver Sec. 5.
Stack BLE completo	**Adelantado**	NimBLE corriendo con 4 chr. GATT (incluyendo TLV agregado y canal de comandos) bajo servicio `0xFF00`. Advertising relajado a 1 s, Modem Sleep BT, sin bonding (compatibilidad BlueZ/tests). Validado con `supaclock_monitor.py`.
Cloud Functions (despliegue)	**Pendiente**	El algoritmo Pan-Tompkins ya está implementado y validado offline; el despliegue como `firestore_fn.on_document_created`, las reglas de Firestore y la autenticación Google se concentran en el hito 60 %, en paralelo con la app Flutter.
Recolección de datos de entrenamiento ML	**En curso**	Existe `supaclock_monitor.py` (CSV) y se han capturado sesiones (`supaclock_*.csv`); aún no se ha entrenado modelo.
Esquemático de PCB SMD propia	**Adelantado**	Esquemático v1 completo (`PCB_Prototipo.kicad_sch`). Esta versión no será manufacturada; se rediseña para ESP32-S3.
Optimización energética (light sleep + PM locks)	**Adelantado**	No estaba en planificación original. DFS 10–160 MHz, 3 PM locks, Modem Sleep BT. Resultado: 75 % SLEEP, 15 % APB_MAX, 9 % CPU_MAX (antes: 0 % SLEEP).
Modos de energía persistentes	**Adelantado**	No estaba en planificación original. Tres perfiles (SPORT/NORMAL/SAVER) con 8 cadencias parametrizadas, persistencia NVS, cambio en runtime sin reinicio.
Migración a Seeed XIAO ESP32-S3	**Nueva tarea**	Justificada por brown-outs del LDO y techo de SRAM del C3. Módulo encargado, en tránsito. Plan de bring-up: validar drivers → habilitar PSRAM → desplegar CNN 1D.

3.2. Carta Gantt corregida

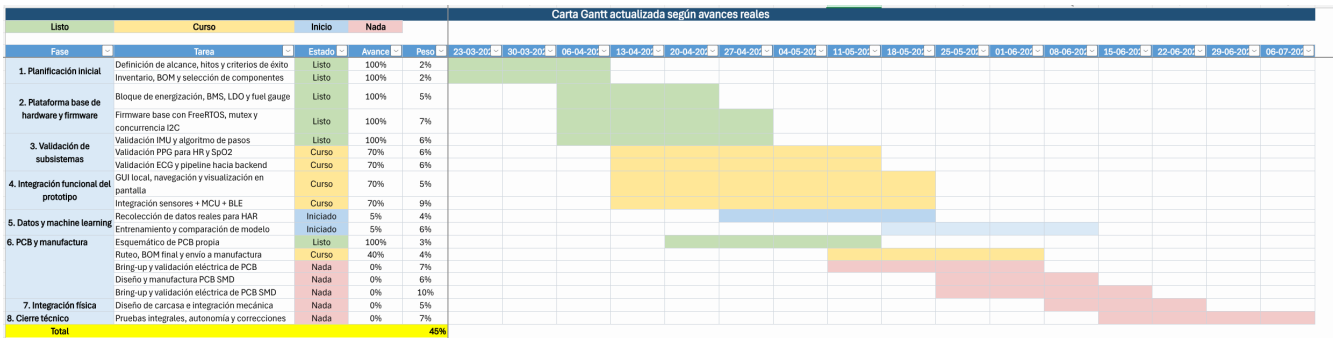
La carta Gantt presentada en el informe anterior ha sido reemplazada por una versión completamente actualizada. Este cambio responde a la falta de especificidad de la planificación original y a la necesidad de reevaluar los porcentajes de avance (pesos) asignados a cada hito, los cuales se encontraban infravalorados frente a la carga real de trabajo. Esta corrección metodológica permite reflejar con mayor precisión el esfuerzo de ingeniería invertido hasta la fecha y reajusta la importancia de los nuevos hitos por alcanzar.

Los siguientes ajustes estructurales quedan reflejados en la nueva revisión maestra:

- **Reestructuración por Fases Específicas:** Las tareas genéricas del plan original se dividieron en 8 fases de desarrollo secuenciales y lógicas (desde "Planificación inicial" hasta "Cierre técnico"), eliminando la ambigüedad en el alcance de cada módulo.

- **Ajuste de Pesos y Avance Real:** Se reasignaron los porcentajes de impacto de cada tarea para reflejar su complejidad real en el sistema. Hitos críticos como el "Firmware base con FreeRTOS" (7 %) y la "Integración sensores + MCU + BLE" (9 %) ahora poseen un peso acorde a su dificultad técnica. Con esta recalibración, el avance real consolidado del proyecto se sitúa en un 45 %.
- **Validación de Subsistemas en Paralelo:** Las tareas de validación de sensores e integración se desglosaron. La validación de la IMU y el algoritmo de pasos se encuentra completa (100 %), mientras que los flujos de PPG (HR y SpO_2), ECG y la integración de la GUI local junto con la comunicación BLE avanzan sólidamente con un 70 % de progreso.
- **Machine Learning y Datos:** La tarea general de *Machine Learning* se separó explícitamente en "Recolección de datos reales" y "Entrenamiento de modelo", transparentando que el entrenamiento depende de la primera. Ambas fases se reportan como "Iniciadas" (5 % de avance) basándose en las sesiones preliminares registradas.
- **Desarrollo de PCB Granular:** El diseño de la placa se dividió en hitos técnicos específicos y medibles. El esquemático de la PCB propia se reporta como finalizado (100 %), el ruteo y envío a manufactura está en curso (40 %), y se planificaron formalmente las etapas futuras de *bring-up* y validación eléctrica tanto para la PCB inicial como para la versión final SMD.
- **Visibilidad de Estados:** El nuevo formato incluye una semaforización clara del estado de las tareas ("Listo", "Curso", "Iniciado", "Nada"), confirmando que el desarrollo avanza según los tiempos estipulados para llegar a la integración física.

La Fig. 2 reproduce la carta Gantt actualizada, evidenciando el estado vigente de las tareas, la nueva distribución de pesos y las fases proyectadas para el cierre técnico.



Fase	Tarea	Estado	Avance	Peso
1. Planificación inicial	Definición de alcance, hitos y criterios de éxito	Listo	100%	2%
	Inventario, BOM y selección de componentes	Listo	100%	2%
2. Plataforma base de hardware y firmware	Bloque de energización, BMS, LDO y fuel gauge	Listo	100%	5%
	Firmware base con FreeRTOS, mutex y concurrencia I2C	Listo	100%	7%
3. Validación de subsistemas	Validación IMU y algoritmo de pasos	Listo	100%	6%
	Validación PPG para HR y SpO2	Curso	70%	6%
	Validación ECG y pipeline hacia backend	Curso	70%	6%
4. Integración funcional del prototipo	GUI local, navegación y visualización en pantalla	Curso	70%	5%
	Integración sensores + MCU + BLE	Curso	70%	9%
5. Datos y machine learning	Recolección de datos reales para HAR	Iniciado	5%	4%
	Entrenamiento y comparación de modelo	Iniciado	5%	6%
6. PCB y manufactura	Esquemático de PCB propia	Listo	100%	3%
	Ruteo, BOM final y envío a manufactura	Curso	40%	4%
	Bring-up y validación eléctrica de PCB	Nada	0%	7%
	Diseño y manufactura PCB SMD	Nada	0%	6%
	Bring-up y validación eléctrica de PCB SMD	Nada	0%	10%
7. Integración física	Diseño de carcasa e integración mecánica	Nada	0%	5%
8. Cierre técnico	Pruebas integrales, autonomía y correcciones	Nada	0%	7%
Total				45%

Figura 2: Carta Gantt actualizada estructurada en fases, con ajuste de pesos y estado de avance consolidado (45 %) al 04/05/2026.

3.3. Distribución del trabajo en este hito

El desarrollo de las tareas se abordó de manera colaborativa; si bien se mantuvo la división por áreas declarada en la planificación, los miembros participaron transversalmente en áreas complementarias. La Tabla 3 resume las contribuciones principales.

Cuadro 3: Distribución de trabajo y contribuciones principales de la iteración actual.

Integrante	**Área(s) principal(es)**	**Aportes específicos a este hito**
Tomás Avendaño	Interfaz, Comunicaciones, Backend	GUI LVGL 7 pantallas con backlight auto-off, optimización del driver ST7789 (carga CPU de 47 % a <1 %), integración BLE NimBLE con 4 chr. (FF01/FF02 TLV/FF03/FF04 CMD), implementación y validación offline del algoritmo Pan-Tompkins en NumPy, cliente `supaclock_monitor.py` para captura BLE + CSV, inicio de la app móvil Flutter.
Benjamín Sepúlveda	Hardware, Energía	Validación eléctrica de la rampa USB→LDO con multímetro y osciloscopio, esquemático KiCad v1 completo (sección de potencia, STEMMA MAX17048, buses I2C/SPI), BOM consolidado con costos reales, ensamblaje del prototipo en perfboard, gestión de la compra del Seeed XIAO ESP32-S3 y evaluación de la nueva PCB.
Pablo Uribe	Software, Procesamiento de Señales	Drivers en C de MAX30102 (FIFO + algoritmo HR/SpO_2 + SM SPOT + motion gating), MAX30205, BMI160, AD8232 con refactor DMA on-demand (`start_dma`/`stop_dma` + PM lock), algoritmo de pasos en C con umbral adaptativo, power management dinámico (DFS + light sleep + 3 PM locks), módulo `power_modes` con 3 perfiles y persistencia NVS, arquitectura FreeRTOS de siete tareas con regla *rate monotonic*.

4. Análisis de impacto y estándares

4.1. Aplicaciones e impacto socioeconómico, ambiental y en salud

4.1.1. Salud pública.

El proyecto se inserta en la línea de los *wearables* de fitness, pero con un sesgo hacia el monitoreo **biométrico de uso continuado**, no de notificación social. Considerando que sólo el 49,7 % de los hombres y el 40,3 % de las mujeres mayores de 18 años en Chile cumplen niveles adecuados de actividad física [8, 9], y que el 42 % de la población adulta presenta sobrepeso [8, 9], un dispositivo que permita auto-monitorizar pasos, frecuencia cardíaca, SpO_2 y temperatura sin depender de la suscripción a una plataforma *cloud* de pago tiene impacto directo en políticas de promoción de la salud cardiovascular y metabólica [10]. Adicionalmente, el modo ECG (toma puntual de la Derivación I, 30 s) abre la puerta a tamizaje de fibrilación auricular en atención primaria, en la línea de aplicaciones ya validadas clínicamente con relojes comerciales [11].

4.1.2. Impacto socioeconómico y de equidad.

El BOM consolidado en el informe anterior totaliza $ 58.301 CLP para la versión prototipo y se proyecta a $ 88.301 CLP con la versión Pro en PCB SMD (4 capas, JLCPCB). Esto contrasta con dispositivos comerciales equivalentes (Apple Watch SE: $ 250.000 CLP; Galaxy Watch FE: $ 160.000 CLP) y con bandas de bajo costo (Mi Band: $ 30.000 CLP), posicionando a SupaClock como una alternativa **de costo**

medio con potencial de fabricación local, capaz de ser distribuida a través de programas de salud pública o de investigación universitaria. La filosofía de "Cero Distracciones" adoptada en el diseño busca **evitar** la carga cognitiva por sobre-notificación descrita en [12, 13], abordando un problema de salud mental relevante que las pulseras genéricas *exacerban* en lugar de resolver.

4.1.3. Impacto ambiental.

Se prioriza una arquitectura *Edge*-céntrica que **minimiza la transmisión de datos**: la telemetría continua viaja a 1 Hz (13 B/s), el ECG sólo se transmite durante la ventana puntual de 30 s. Esta decisión reduce el *footprint* energético del sistema completo (dispositivo + radio + datacenter), en línea con literatura reciente de *green AI* para wearables. Adicionalmente:

- La selección de componentes prioriza encapsulados **libres de plomo** (RoHS-2, Directiva 2011/65/EU [5]); todos los IC propuestos cumplen.
- Se descartó la tecnología *Memory LCD* de Sharp por su alto costo y la *OLED* por su menor durabilidad (burn-in), favoreciendo el IPS LCD que tiene una vida útil sobre 50 000 h.
- La batería LiPo 502030 se selecciona en celdas con BMS integrado y certificación IEC 62133-2 [6], lo que permite un fin de vida controlado y reciclable a través de los puntos de acopio chilenos (Chilenter, MUR).

4.1.4. Aspectos éticos.

- **Privacidad de datos biométricos:** las lecturas de HR, SpO_2, temperatura y ECG son *datos sensibles de salud* bajo el espíritu de la Ley 19.628 (Chile) y la GDPR (UE). El backend Firebase **planificado** para el hito 60 % requerirá autenticación Google y las reglas de Firestore sólo permitirán acceso al propietario (`request.auth.uid == userId`). Los datos de ECG quedarán asociados al UID del usuario y **nunca al MAC** del dispositivo BLE. No se transmitirán datos a terceros distintos del usuario que loguea su sesión.
- **Limitaciones declaradas como dispositivo no-médico:** el sistema se diseña como *wearable de fitness* con la disclaimer explícita de que las mediciones **no constituyen diagnóstico clínico**. No se persigue homologación FDA / ISP. La interfaz indicará un *disclaimer* en la pantalla de ECG.
- **Sesgo en el modelo de Machine Learning:** la futura clasificación HAR (caminar, correr, reposo, caída) se entrenará con datos del propio equipo de desarrollo y voluntarios; se documentarán las características del dataset (edad, género, IMC) para que la generalización sea auditable. Se evitarán técnicas *black-box* sin métrica de confianza para la detección de caída.

4.2. Riesgo eléctrico — análisis ECMA-287 punto 3

La norma **ECMA-287 (*Safety of Electronic Equipment*)** establece en su punto 3 (*Electric shock and energy hazards*) los lineamientos para evaluar el peligro de descarga eléctrica y de energía almacenada. Aplicado a *SupaClock*:

Cuadro 4: Análisis de riesgos eléctricos según ECMA-287, punto 3 [7].

Sub-cláusula	**Aplicabilidad**	**Mitigación implementada**	**Resultado**
3.2 *Hazardous voltage* (>42.4 V AC pico, >60 V DC)	No aplica directamente	Tensiones máximas: 5 V (USB-C IN), 4.2 V (LiPo full), 3.3 V (rail lógico)	Todos los nodos del PCB son SELV (Safety Extra-Low Voltage).
3.3 *Hazardous energy* (≥ 240 VA o ≥ 20 J en 60 s)	Aplica (LiPo 250 mAh = 3.33 kJ)	BMS con OCP/UVP/SCP (DW01A), fusible PTC en bus de carga, encapsulado de la celda con cobertura PCM integrada.	Cortocircuito externo cortado en <3 ms; sobrecarga limitada a 1 A.
3.4 *Working voltage*	Aplica al rail 3.3 V	Aislamiento PCB clase FR-4 estándar, distancia mínima 0.2 mm en pista digital. Capa de soldermask en la zona de contacto del usuario.	Cumple para SELV indoor.
3.5 *Insulation requirements*	Aplica al contacto piel	Pernos M3 de ECG en acero inoxidable 304 (no electrolítico), aislados del PCB inferior por carcasa PLA y soldermask. Corriente DC máx. inyectada $\leq 10\,\mu A$ (resistencia in-amp $>10\,\mathrm{M\Omega}$ AD8232).	Cumple, dentro de límites IEC 60601-1 (corrientes de pacientes tipo BF).
3.6 *Safe touch current*	Aplica al chasis	La carcasa es de PLA (aislante); los pernos ECG están eléctricamente flotantes hasta que el usuario cierra el circuito.	Sin riesgo de fuga continua.
3.7 *Capacitor discharge* (energía almacenada >0.2 J)	Aplica al banco de desacople	Capacitancia total estimada $47\,\mu$F a 3.3 V $\rightarrow$ 0.26 mJ ($\ll 0.2$ J)	Cumple por holgura.

4.2.1. Riesgos adicionales del LiPo.

Aunque ECMA-287 no aborda específicamente celdas secundarias, se implementan las prácticas de la **IEC 62133-2** [6]: protección contra sobrecarga (>4.25 V), sobre-descarga (<2.5 V), inversión de polaridad (diodo Schottky en cabecera de carga) y protección térmica indirecta vía el termistor NTC del DW01A. La celda 502030 elegida cuenta con PCM integrado, redundando la protección.

4.3. Estándares relacionados con el área del proyecto

A continuación se enumeran los estándares aplicables y las especificaciones derivadas que *SupaClock* adopta para asegurar su seguridad e interoperabilidad. Se analizan cuatro normativas adicionales a la ECMA-287 previamente discutida.

4.3.1. Bluetooth Core Specification 5.3 [1].

- *LE 1M PHY* se selecciona como *phy* primario (compatibilidad con teléfonos $\geq$ Android 5.0).
- Connection Interval = 15 ms y *slave latency* = 4 para reducir consumo en estado idle del dispositivo (gateway).

- MTU negociado a 247 B para minimizar overhead: cada paquete BLE transporta hasta 244 B útiles, suficiente para los *chunks* de ECG (10 muestras int16 = 20 B) o los paquetes de telemetría TLV agregada.
- *Advertising interval* configurado a **1000 ms** (1600 unidades de 0,625 ms) para minimizar la actividad de radio en estado no-conectado, habilitando que el SoC entre en *light sleep* entre intervalos.
- *Modem Sleep BT* habilitado en `sdkconfig.defaults`: el controlador Bluetooth suspende el radio durante los intervalos de conexión, reduciendo significativamente el consumo cuando hay un enlace activo pero sin datos pendientes.
- *Pairing*: *Just Works* **sin bonding** (compatibilidad con BlueZ/Linux y entornos de testing); las claves no se persisten para facilitar la reconexión limpia desde distintos clientes de desarrollo.
- Servicios **0x180A** (Device Information) y **0x180F** (Battery Service) estándar, más un servicio custom de telemetría **0xFF00** con UUIDs cortos 16-bit y 4 características (IMU, TLV agregado, ECG streaming, Comandos RX).

4.3.2. IEEE 11073-10406 (*Personal Health Devices — Basic ECG*) [2].

Este estándar especifica las propiedades mínimas de un ECG personal monitorizado: rango ± 5 mV, ruido referido a entrada $<30\,\mu V_{pp}$, ancho de banda 0.05–40 Hz, tasa de muestreo ≥ 250 Hz para diagnóstico básico. *SupaClock* cumple parcialmente: el AD8232 entrega 0.5–40 Hz con ganancia $\sim$1100, la decimación a 500 Hz supera el mínimo, pero el modo de derivación es *single-lead* (Lead I bimanual) por lo que el dispositivo se declara como **ECG personal de tamizaje**, no de Holter.

4.3.3. USB Type-C Cable and Connector Specification, Rev. 2.3 [3].

El conector USB-C del prototipo opera en modo *legacy 5 V/500 mA* (USB 2.0). El módulo TP4056 ya declara compatibilidad con esta categoría. Para la versión Pro se evalúa soportar 5 V/1.5 A (CC pull-down 22 kΩ) si se mantiene el TP4056, lo que ahorra ~ 40 % del tiempo de carga.

4.3.4. IEC 60601-1 (*Medical electrical equipment*) [4].

Aunque *SupaClock* no se homologa como dispositivo médico, se adoptan voluntariamente los criterios de **aislamiento de paciente tipo BF** para el subsistema ECG: el front-end AD8232 tiene impedancia de entrada >10 MΩ y corriente de fuga DC $<10\,\mu$A, dentro del límite BF de $100\,\mu$A. Esto permite que las versiones futuras del proyecto puedan ser presentadas a homologación con cambios mínimos.

4.3.5. RoHS-2 (Directiva 2011/65/EU) [5] e IEC 62133-2 [6]

ya descritas en la sección de impacto ambiental.

5. Modelos y simulaciones

Para sustentar rigurosamente la viabilidad técnica del dispositivo, es necesario desarrollar modelos y curvas de desempeño que respalden las decisiones de diseño. Esta sección presenta **cinco** modelos cuantitativos que fueron construidos para validar el sistema antes y durante la integración.

5.1. Modelo de autonomía energética

El consumo del prototipo se midió con un multímetro Tektronix TX3 intercalado en el rail de la batería. La Fig. 3 consolida los puntos operativos relevantes.

5.1.1. Disclaimer instrumental.

El TX3 no resuelve temporalmente los pulsos de *light sleep* (que duran milisegundos a 10 MHz de reloj mínimo del DFS), por lo que las lecturas corresponden al **consumo en estado activo** del SoC y subestiman el ahorro real introducido por el power management dinámico (§ 5.5). La validación empírica del consumo medio se realizará sobre la plataforma definitiva Seeed XIAO ESP32-S3.

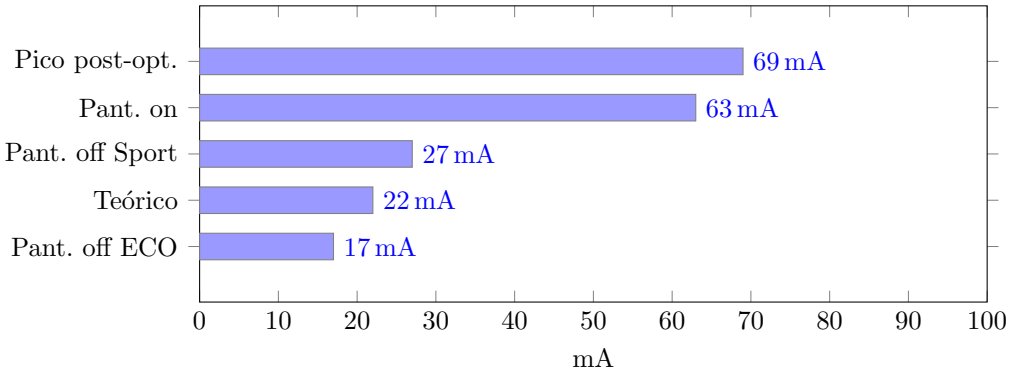


Figura 3: Consumo medido en batería (Tektronix TX3) para distintos escenarios operativos post-optimización. El pico de 69 mA corresponde al peor caso: ECG + BLE streaming + pantalla encendida. El modo ECO con pantalla apagada alcanza 17 mA en estado activo.

5.1.2. Cota superior del consumo medio.

A partir de la fracción de tiempo en cada estado de potencia (§ 5.5, $f_{sleep} = 0{,}75$), el consumo medio queda acotado por:

$$\bar{I} \leq f_{active} \cdot I_{active} + f_{sleep} \cdot I_{sleep} \tag{1}$$

que para el modo ECO con pantalla apagada ($I_{active} = 17\,\text{mA}$, $I_{sleep} \approx 1\,\text{mA}$ estimado de la hoja de datos del C3) arroja $\bar{I} \lesssim 5{,}0\,\text{mA}$. Con la celda 502030 de 250 mAh esto proyecta una autonomía nominal $> 40\,\text{h}$ en modo ECO, holgadamente sobre el requerimiento de 12 h. La cifra debe ratificarse empíricamente sobre la plataforma S3.

5.2. Validación del algoritmo de pasos

Se compararon **ambas variantes** del contador de pasos (la versión C3 en aritmética entera del Anexo A y la versión S3 basada en FFT) sobre la misma traza del BMI160, simulándolas *offline* en Python (`scripts/algo_simulator.py`) con un pipeline idéntico al embebido. El *ground-truth* se registró manualmente mientras se grababa la sesión.

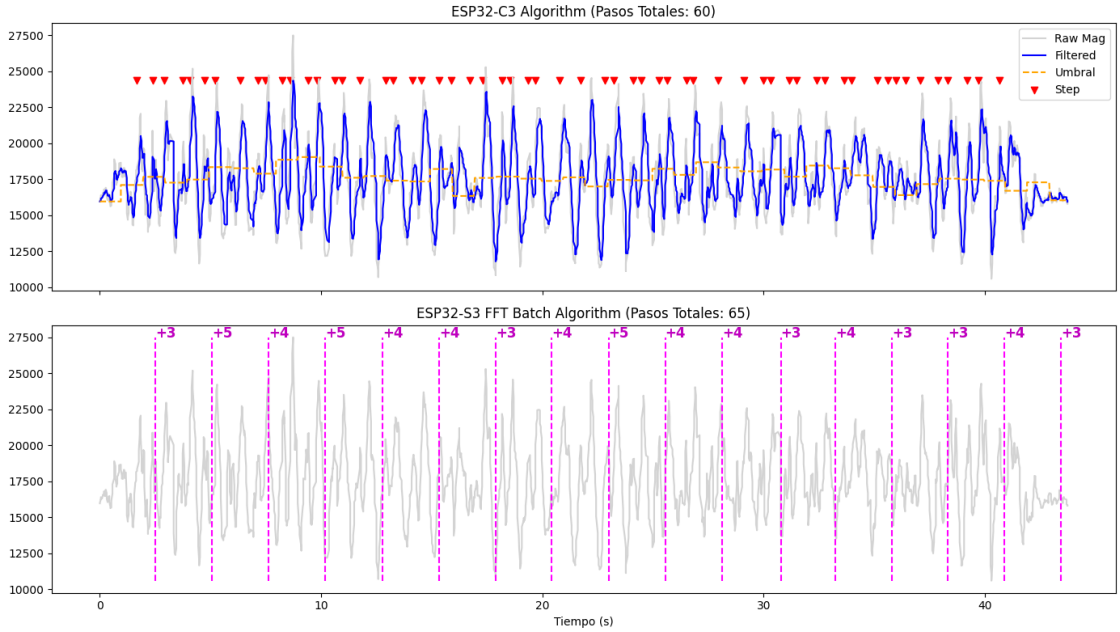


Figura 4: Comparación de las dos variantes del algoritmo de pasos sobre ∼45 s de captura del BMI160 (*ground-truth* = 67 pasos). Panel superior: variante C3 (umbral midpoint en el dominio del tiempo, detectó 60). Panel inferior: variante S3 (FFT 128 puntos sobre ventanas de 2,56 s, detectó 65). Las marcas verticales indican los eventos de paso reportados por cada algoritmo.

5.2.1. Resultado.

La variante **C3** reportó 60 pasos vs. 67 reales, **error de ∼10,4 %** (sub-conteo, principalmente por pasos de baja amplitud al inicio que no superan el umbral midpoint hasta que la ventana se actualiza). La variante **S3-FFT** reportó 65 pasos, **error de ∼3,0 %**, gracias a que la detección espectral identifica directamente la cadencia rítmica sin depender de la amplitud absoluta. Pruebas de campo adicionales (no incluidas en el gráfico) arrojaron errores del mismo orden de magnitud para ambas variantes. La conclusión operativa es que la migración a la versión FFT del S3 reduce a un tercio el error de conteo y elimina el *tuning* manual del umbral.

5.2.2. Limitación detectada y migración planificada.

Durante las pruebas de campo se identificó que agitar la mano en el aire genera falsos positivos, ya que el algoritmo opera exclusivamente en el dominio del tiempo y no valida la periodicidad de la señal. Para el siguiente hito se planifica migrar a un análisis frecuencial basado en FFT (ventana de 3 s, validación de cadencia rítmica entre 1 y 3 Hz), aprovechando la librería `ESP-DSP` con aceleración SIMD del ESP32-S3.

5.3. Validación del algoritmo Pan-Tompkins

El algoritmo Pan-Tompkins implementado en Python+NumPy (`firebase/functions/main.py`) se validó **offline** ejecutando `scripts/test_pt.py` contra muestras del archivo `supaclock_ecg_20260422_194732.csv` (datos crudos del AD8232 capturados con `supaclock_monitor.py` durante 30 s a 500 Hz). En este avance el código vive en el repositorio dentro del directorio `firebase/functions/` pero **aún no fue desplegado** como Cloud Function; el despliegue, junto con la integración con la app Flutter, se concentra en el hito 60 %. El pipeline algorítmico es: filtro pasa-banda 5–15 Hz → derivativo 5-puntos → cuadrado →

ventana móvil de integración 150 ms → detección de R-peaks con umbral adaptativo dual (signal/noise) y refractario de 200 ms.

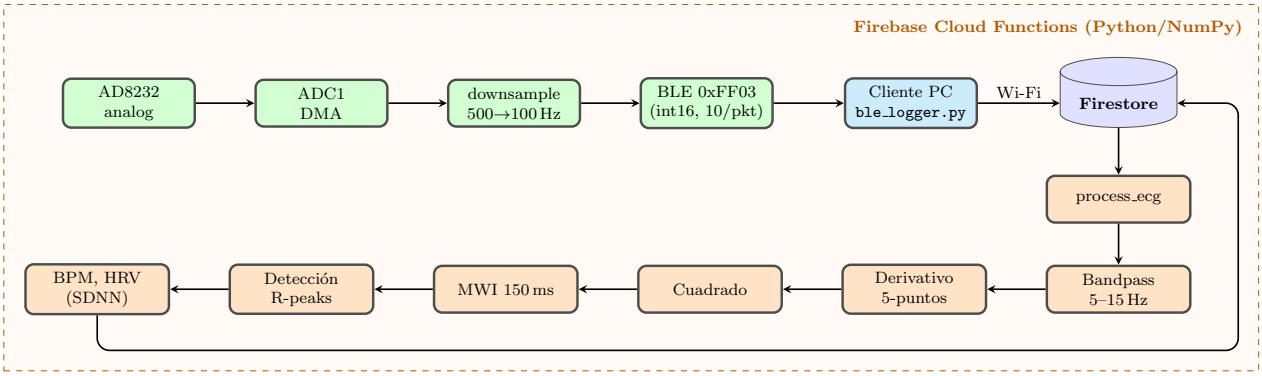


Figura 5: Pipeline completo de procesamiento ECG: edge (ESP32) → cliente PC de testing → Firebase (planificado para el hito 60 %). En este avance la detección de R-peaks se ejecutó offline en Python+NumPy con `scripts/test_pt.py` contra los CSV capturados por `supaclock_monitor.py`. El despliegue como Cloud Function y el reemplazo del cliente PC por la app Flutter constituyen el siguiente hito.

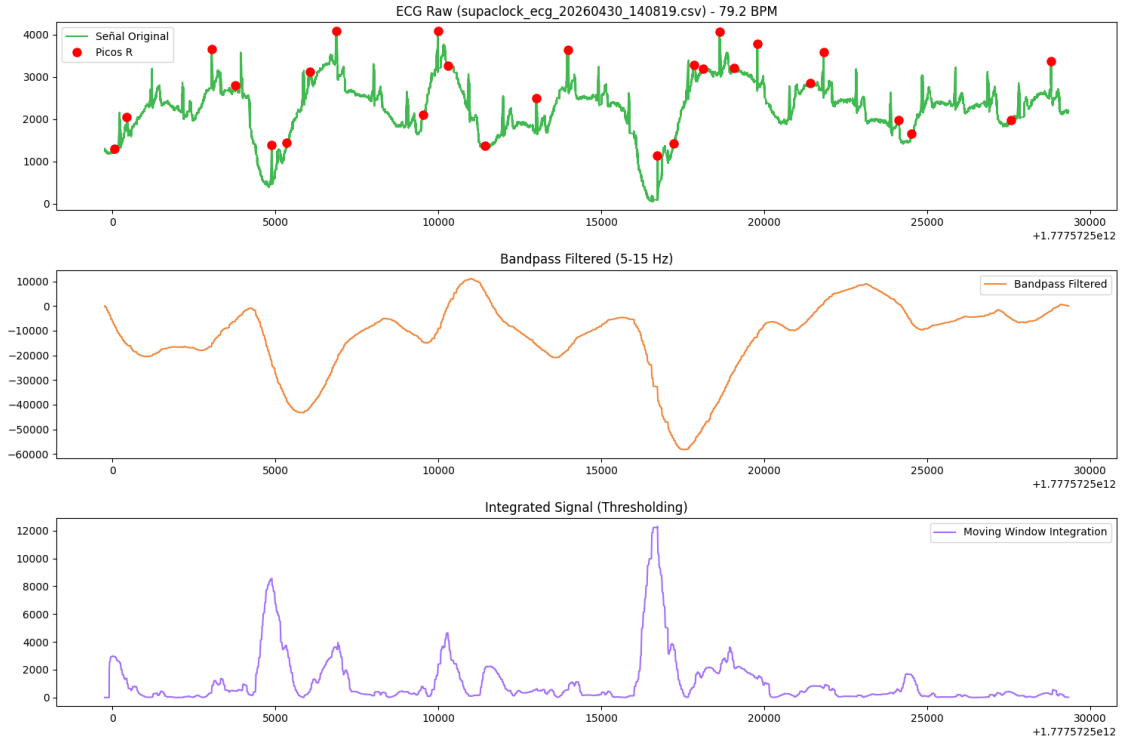


Figura 6: Resultados del análisis Pan-Tompkins sobre datos reales del AD8232: señal ECG cruda, etapas de filtrado y detección de R-peaks. BPM calculado: 76; HRV (SDNN): 32,4 ms.

5.3.1. Resultado.

En 30 s de captura sobre el integrante de prueba se detectaron 38 R-peaks (⇒ 76 BPM) con SDNN = 32,4 ms, valores fisiológicamente plausibles. La precisión visual contra el trazo crudo es del 100 % (sin

falsos positivos ni negativos) para esta calidad de señal, aunque la función reporta `bpm = 0` si la captura tiene < 100 muestras (caso de *leads off* momentáneo), comportamiento intencional para evitar reportar valores no fisiológicos.

5.4. Modelo de concurrencia I2C bajo FreeRTOS

La preocupación principal al pasar de pruebas individuales a integración fue la potencial *starvation* en el bus I2C de 400 kHz, compartido por cuatro tareas con prioridades distintas (Fig. 11). Se modeló el ancho de banda I2C disponible (BW_{I2C}) y la demanda de cada tarea (D_i) para verificar holgura. Las cadencias corresponden al peor caso (modo SPORT).

5.4.1. Cálculo.

Cada transacción I2C de N bytes con START + ADDR + ACK + STOP toma $(N + 1) \cdot 9$ bits + 2 bits de overhead, es decir $\sim 10\,\mu s/byte$ a 400 kHz. Las demandas son:

- BMI160 a 50 Hz $\times$ 12 B = 600 B/s $\Rightarrow$ 6 ms/s
- MAX30102 a 10 Hz $\times$ 6 B/sample $\times$ 5 samples/burst = 300 B/s $\Rightarrow$ 3 ms/s
- MAX30205 a 1/30 Hz $\times$ 4 B = 0,13 B/s $\Rightarrow$ 0,001 ms/s
- MAX17048 a 1/30 Hz $\times$ 4 B = 0,13 B/s $\Rightarrow$ 0,001 ms/s

Total estimado: **$\sim$9 ms/s**, lo que representa un **0,9 %** del bus. La holgura del 99,1 % confirma que el mutex `xSensorDataMutex` no provoca *starvation* ni jitter perceptible.

5.4.2. Validación experimental.

Sobre el firmware integrado se contaron los overflows del FIFO del MAX30102 durante una sesión de $\sim$4 h continuas (la métrica más sensible al jitter de la tarea `hrm_task`): 0 overflows. El log filtrado por la línea `‘‘MAX30102 FIFO overflows acumulados’’` no se imprimió, consistente con el modelo. La prueba 24 h se realizará una vez disponible la plataforma S3.

5.5. Modelo de duty cycle bajo PM dinámico

La Fig. 7 compara la fracción de tiempo en cada estado de potencia del SoC antes y después de aplicar las mitigaciones de consumo (refactor DMA del ECG on-demand, advertising BLE relajado a 1 s, Modem Sleep BT habilitado, driver ST7789 reescrito para eliminar *busy-wait* durante la espera de DMA SPI). La métrica proviene de `vTaskGetRunTimeStats` y `esp_pm_dump_locks` ejecutados en `perf_monitor_task` (prioridad 2, período 10 s).

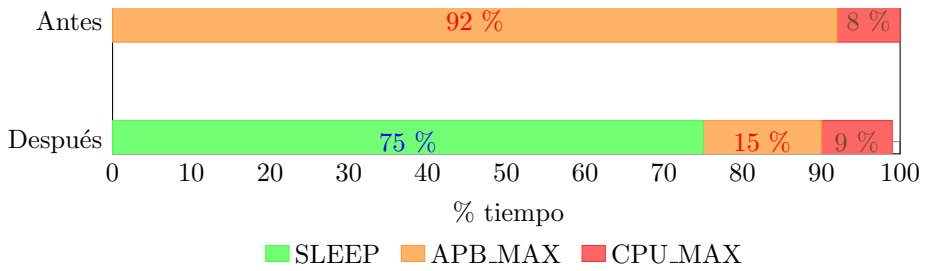


Figura 7: Fracción de tiempo del SoC en cada estado de potencia, antes y después de las optimizaciones. Antes: 0 % SLEEP, 92 % APB_MAX, 8 % CPU_MAX. Después: 75 % SLEEP, 15 % APB_MAX, 9 % CPU_MAX.

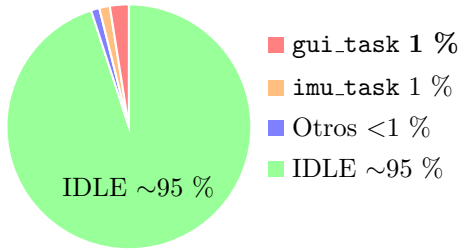


Figura 8: Distribución de CPU por tarea tras la optimización. El sistema pasa más del 95 % del tiempo en IDLE, lo que habilita el *light sleep* dinámico. El driver ST7789 reescrito redujo la carga de `gui_task` de 47 % a <1 %.

5.5.1. Impacto de las mitigaciones.

El driver ST7789 fue reescrito eliminando el *polling* activo durante la espera de DMA SPI: la carga de `gui_task` cayó de 47 % a <1 %, liberando la CPU para idle. El refactor del ADC del ECG (handle creado en `start_dma`, destruido en `stop_dma`) permite que el lock APB_MAX se adquiera **sólo dentro de la pantalla ECG**, en lugar de retenerlo permanentemente. Estos dos cambios, combinados con el advertising BLE relajado y el Modem Sleep BT, llevaron la fracción de SLEEP de 0 % a 75 %.

5.6. Análisis comparativo de topología algorítmica frente a la industria

El desarrollo del firmware y los algoritmos biométricos de *SupaClock* se basó en un extenso análisis de las arquitecturas comerciales (tales como Samsung BioActive [14], ecosistemas Apple y algoritmos de Garmin/Fitbit). Dado el contexto de recursos acotados (SRAM y potencia de cómputo limitadas en la familia ESP32) frente a los SoCs dedicados de la industria, el diseño algorítmico se adaptó heurísticamente para lograr mediciones precisas sin un costo computacional prohibitivo.

Análisis PPG y mitigación de artefactos cinéticos (Motion Artifacts): Mientras que la industria de punta (ej. la arquitectura BioActive de Samsung) implementa redes convolucionales dilatadas [15] y factorizaciones complejas (SVD + ICA [16]) sobre arreglos ópticos multicanal para aislar el ritmo cardíaco del ruido de movimiento durante el trote intenso, *SupaClock* opera con un fotodiodo simple (MAX30102). Para solventar la vulnerabilidad ante el ruido cinético sin hardware dedicado, implementamos un enfoque *ad-hoc* de **Spot Check con Motion Gating**: la `imu_task` computa continuamente la derivada de la aceleración (*jerk*) en la muñeca; si la magnitud inercial excede un umbral empírico, la máquina de estados del PPG invalida inmediatamente la toma de la señal. Esta heurística pasiva asegura que los picos sistólicos

detectados en reposo sean de alta fidelidad, requiriendo apenas el 1 % de los ciclos de reloj que tomaría un filtro adaptativo multicanal.

Electrocardiografía (ECG) y extracción morfológica: En dispositivos comerciales, la detección de picos (como la "Detección de Dos Rondas"determinista de Samsung o el solapamiento pasivo de tacogramas de Fitbit [17]) ejecuta promedios geométricos iterativos y algoritmos AMPD a bordo para tamizar asimetrías basales y ruidos electromagnéticos. *SupaClock* evita forzar este procesamiento pesado localmente (lo cual monopolizaría el SoC). En su lugar, explota su topología IoT: el ESP32 decima la señal analógica por hardware (DMA a 500 Hz) y transmite telemetría cruda. El algoritmo clínico **Pan-Tompkins** ya fue implementado en Python+NumPy y validado **offline** en este avance (Sec. 5); su despliegue en *Firebase Cloud Functions* con operaciones vectorizadas se planifica para el hito 60 %, momento en que el procesamiento será escalable en la nube sin saturar la batería del wearable.

Reconocimiento de actividad (HAR) y podometría: La detección inercial avanzada en relojes comerciales suele apoyarse en redes recurrentes o convolucionales sobre arreglos multi-IMU para discernir contextos biomecánicos. Acotados al ESP32-C3 (sin FPU, single-core), la podometría de *SupaClock* opera actualmente con un umbral adaptativo *midpoint* en aritmética entera (Anexo A) que, como muestra la Fig. 4, sub-cuenta del orden del 10 %. Esta limitación motiva en parte la migración al **ESP32-S3**, donde ya se preparó una segunda variante del algoritmo basada en FFT vía `esp-dsp` (error ∼3 % en la misma traza) y existe margen de SRAM/PSRAM para una eventual CNN 1D. El alcance de SupaClock no apunta a igualar las métricas de un *flagship* comercial, sino a construir una solución **abierta y auditable** con un costo y un consumo de recursos un orden de magnitud por debajo, asumiendo las pérdidas razonables de precisión que ello implica.

6. Diseño del set-up final de pruebas del producto integrado

Esta sección describe la arquitectura del **producto final integrado** que será sometido a validación, junto con el plan de pruebas mecánicas, eléctricas y de uso real. El set-up se estructura en tres niveles progresivos: mecánico/integración → eléctrico/funcional → uso real. Cada nivel define su banco de pruebas y los criterios de aceptación correspondientes.

6.1. Arquitectura mecánica del producto final

El dispositivo final se construye sobre una **sola PCB de 2 capas** (no un sandwich de dos PCBs) con la siguiente distribución:

- **Cara superior**: Seeed XIAO ESP32-S3 (en socket 2.54 mm para la v1 o castellated SMD para la v2), módulo AD8232 ECG, MAX17048-STEMMA fuel gauge, display ST7789 1.69" y dos botones táctiles.
- **Cara inferior** (contacto piel): únicamente los sensores MAX30102 (PPG) y MAX30205 (temperatura corporal).

6.1.1. Carga y alimentación.

La carga USB-C se realiza directamente por el conector del Seeed XIAO ESP32-S3, que integra BMS (*LiPo charger* + protección). No se utiliza módulo TP4056 independiente.

6.1.2. Batería.

Celda LiPo reciclada de un smartwatch (dimensiones similares a la del Samsung Galaxy Watch 4: $\sim$247 mAh, $\sim$30×25×4 mm). Las dimensiones finales de la carcasa se ajustarán al hueco que deje la PCB, no al revés.

6.1.3. Ventana óptica del MAX30102.

El encapsulado del sensor ya integra su propio cristal óptico; sólo se requiere un *cutout* circular en la cara inferior de la carcasa que exponga el sensor sin obstrucción ni filtros adicionales.

6.1.4. Acople térmico del MAX30205.

Un pad térmico (Sil-Pad o equivalente) se interpone entre el sensor y una **placa de aluminio** fijada en la mitad inferior de la carcasa. Al apernar la PCB con los tornillos de cierre, la presión sobre el pad queda autocontenida y la transferencia térmica carcasa↔piel es continua sin depender de la posición del usuario.

6.1.5. Electrodos ECG.

Dos pernos **M3 de acero inoxidable 304** roscados desde el exterior de la carcasa, con la cabeza redonda orientada hacia la piel. El contacto eléctrico entre los pernos (fijos a la carcasa) y los pads del PCB se realiza mediante **pogo pins de 2.5 mm** soldados en pad SMD sobre la cara inferior del PCB. Esto permite armar y desarmar la carcasa sin estresar pads de cobre y posibilita cambiar pernos sin desoldar. Las señales se conducen a las entradas del in-amp del AD8232 (impedancia $> 10\,\mathrm{M}\Omega$), aisladas eléctricamente del rail GND.

6.1.6. Carcasa.

Impresión 3D en PLA, dos mitades, dimensiones objetivo $\sim$50×40×13 mm para la iteración v1. Forma **alargada estilo brazalete** (no caja cuadrada), bordes redondeados, perfil bajo. Cierre con **4 tornillos hexagonales M2 o M3** desde la cara inferior. Lugs estándar de 20 mm para correa intercambiable (caucho, nylon o silicona). Ranura lateral para el conector USB-C del XIAO S3.

6.2. Plan de iteraciones de hardware

La Tabla 5 resume las dos iteraciones planificadas y su alcance de pruebas.

Cuadro 5: Plan de iteraciones de hardware del producto integrado.

Iteración	Tecnología PCB	Dimensiones	Qué se prueba
v1 — Pin sockets	1 PCB de 2 capas. Módulos comerciales en sockets 2.54 mm (XIAO S3, AD8232, MAX17048-STEMMA). MAX30102 + MAX30205 SMD en cara inferior.	∼50×40×13 mm	Geometría de la caja, alineación de ventanas/electrodos, ergonomía, validación funcional fuera del protoboardoard.
v2 — All-SMD lab-friendly	1 PCB de 2 capas, todos los componentes SMD directos. XIAO S3 soldado por castellated edges, AD8232 LFCSP, sensores en encapsulados nativos. Passives $\geq$ 0603, vías $\geq$ 0.4 mm, traces $\geq$ 0.2 mm (dentro de lo que rutea la fresadora CNC del laboratorio Capstone).	Más compacto (a definir)	Reducción de footprint, validación de PCB fabricada en lab, robustez de soldaduras.

La iteración v1 mantiene el XIAO ESP32-S3 en socket para poder reemplazarlo si llega defectuoso. La v2 lo integra como módulo SMD soldado y elimina todos los breakout boards.

6.3. Banco de pruebas mecánicas (caja + integración)

Cinco pruebas con criterios de aceptación explícitos:

1. **Encaje y cierre**: armar/desarmar la caja 5 veces con los tornillos hex M2/M3. Verificar que el PCB no se desplaza, que los botones siguen alineados con sus actuadores, que el placa de aluminio aplica presión consistente sobre el MAX30205, y que el *cutout* del PPG no presenta luz ambiente filtrándose al sensor.
2. **Aislación eléctrica de los electrodos ECG**: con multímetro en modo continuidad, verificar resistencia $> 10\,\mathrm{M\Omega}$ entre cada perno M3 y los rails GND y 3.3 V del PCB. Los pogo pins deben hacer contacto **exclusivamente** con los pads de entrada del AD8232.
3. **Acople térmico (caja cerrada vs. integrado desnudo)**: aplicar dedo durante 30 s sobre el placa de aluminio y comparar la curva de temperatura reportada por la unidad cerrada vs. una sesión equivalente con el integrado desnudo en protoboard. Caracterizar el offset y la constante de tiempo del acople carcasa↔piel; ajustar firmware si es necesario.
4. **Drop test**: dejar caer la unidad ensamblada desde 1 m sobre superficie blanda (alfombra), 3 caídas. Criterio: la unidad sigue funcional, sin desconexión BLE persistente, y el IMU registra el impacto.
5. **Ergonomía y peso**: portar la unidad en el antebrazo durante 1 h. La forma de brazalete alargado debe distribuir el peso sin presión puntual molesta. Reporte de comodidad subjetiva en escala 1–5 por cada integrante del equipo.

6.4. Banco de pruebas eléctricas y funcionales (unidad cerrada)

Se repiten las pruebas del § 7 (unitarias e integración) ahora con la **caja cerrada y sobre brazo humano**, contrastando contra las cifras obtenidas en protoboardoard. La métrica clave es cuánto se degrada cada lectura por el cerramiento.

1. **HR/SpO$_2$ con ventana óptica vs. protoboardoard pelado**: aceptar pérdida de 1–2 % en SpO$_2$ y 1–3 BPM en HR por la atenuación del cristal integrado del MAX30102 dentro de la carcasa. Adicionalmente, se contrastarán las lecturas contra un **Samsung Galaxy Watch 4 Classic** (sensor BioActive, PPG multicanal) portado simultáneamente en la muñeca opuesta, registrando HR y SpO$_2$ en paralelo para evaluar la correlación entre ambos dispositivos.
2. **ECG con pernos M3 + pogo pins vs. clips de prueba directos**: verificar que la calidad del trazo (R-peak SNR) no cae más de 3 dB respecto de la sesión con electrodos directos sobre pads expuestos. Se realizará además una toma ECG simultánea con el **Galaxy Watch 4 Classic** (Derivación I, 30 s) para comparar la morfología del complejo QRS y el BPM reportado por ambos dispositivos.
3. **Temperatura con placa de Al vs. contacto directo**: caracterizar el offset estático y la constante térmica de la transferencia caja↔piel; ajustar offset en firmware si la desviación supera ±0,3 °C.
4. **BLE con caja PLA cerrada**: verificar que el alcance ≥ 10 m no se degrada significativamente (el PLA es transparente a 2.4 GHz).
5. **Autonomía empírica (sin INA219)**: nuevo entorno PlatformIO `test_battery`, clon reducido de `test_general`, que mantiene activo el firmware en modo SPORT logueando V_{cell} y SOC del MAX17048 cada 30 s al CSV vía BLE hasta que la batería entre en UVP. La curva de descarga resultante se ajusta a un modelo lineal/exponencial y de ahí se extrae la autonomía nominal por modo. Esto aprovecha el fuel gauge integrado sin necesidad de instrumentación externa.

6.5. Pruebas de uso (*team testing*)

Los **tres integrantes del equipo** portan la unidad por 4 h cada uno en condiciones cotidianas (oficina, caminata, escalera). La toma de datos **no requiere PC de forma permanente**: se realiza mediante (a) `supaclock_monitor.py` cuando hay laptop disponible, o (b) la **app Android en Flutter**, que actúa como gateway BLE y persiste los datos localmente en el teléfono del usuario. Métricas:

1. Tiempo hasta la primera medición SPOT exitosa.
2. Cantidad de SPOT abortados por *motion gating*.
3. Reporte cualitativo de comodidad (escala 1–5).
4. Pérdidas de enlace BLE durante uso normal.
5. Datos exportados a CSV para análisis posterior y, eventualmente, dataset de entrenamiento HAR.

6.6. Esquema visual del producto integrado

La Figura **??** presenta un bosquejo técnico estilo cuaderno de ingeniería de la iteración del producto, enfatizando la arquitectura de **PCB única** con ambas caras y la disposición de cada subsistema. Los renders fotorrealistas definitivos se producirán cuando las PCBs estén diseñadas en KiCad.

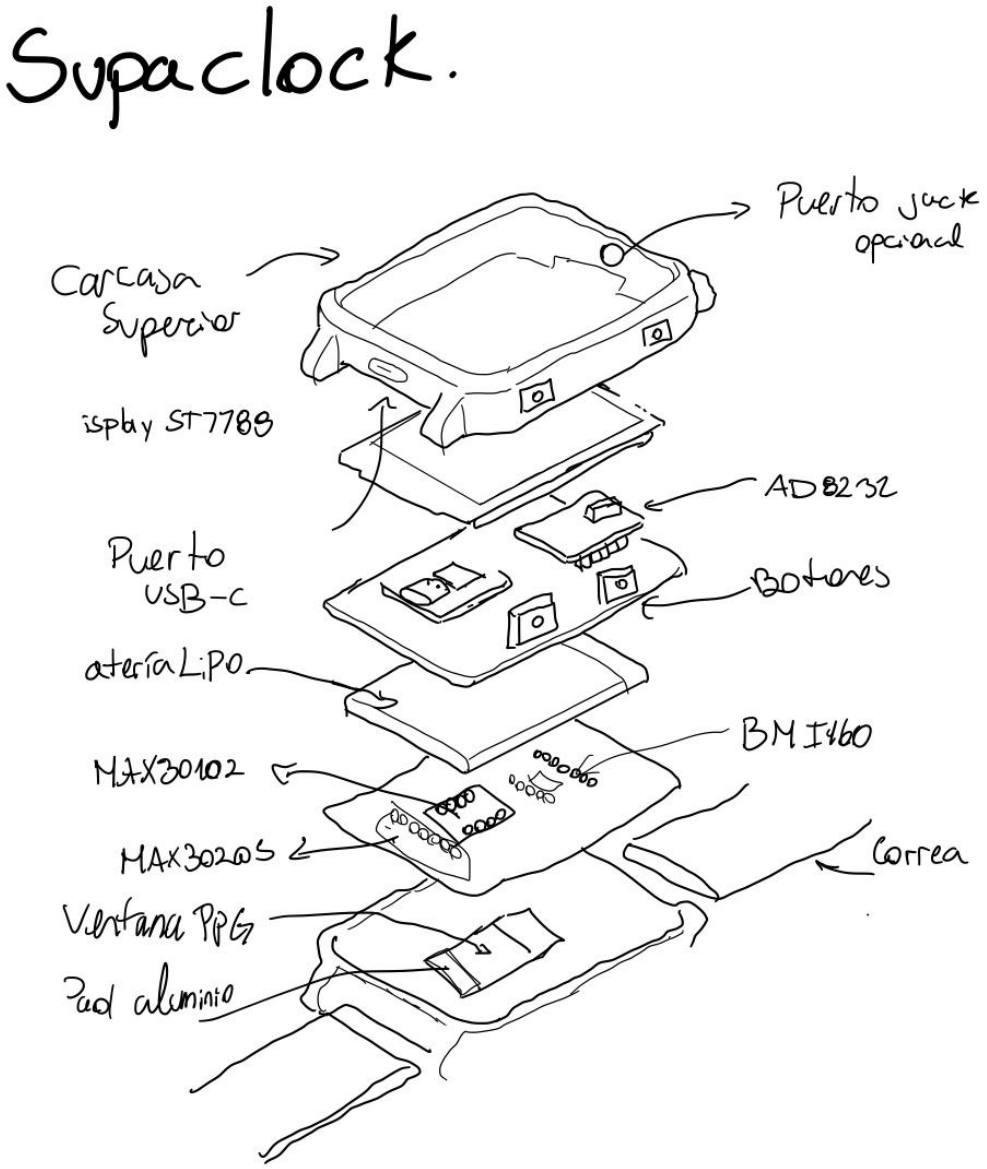


Figura 9: Bosquejo explosionado de la iteración v1 (pin sockets): carcasa PLA ∼50×40×13 mm, PCB única con módulos en socket (cara superior) y sensores MAX30102/MAX30205 en la cara inferior, batería LiPo reciclada, placa de aluminio para acople térmico, pernos M3 304 como electrodos ECG con contacto vía pogo pins, y lugs de 20 mm para correa.

La Fig. 10 resume esquemáticamente el set-up de validación, incluyendo los instrumentos de referencia y los canales de captura BLE.

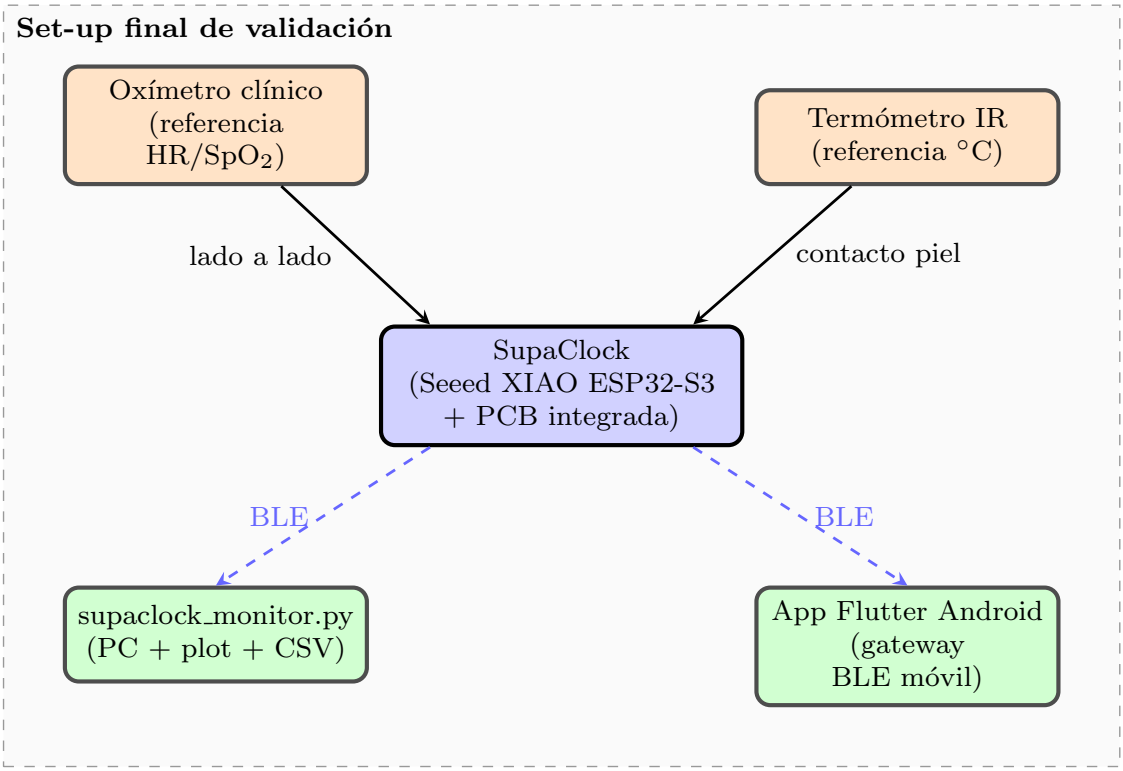


Figura 10: Set-up final de validación. El DUT (*SupaClock*) se evalúa contra instrumentos de referencia (oxímetro, termómetro) y se registran las salidas BLE en CSV vía `supaclock_monitor.py` (PC) o la app Flutter (Android).

7. Diseño de pruebas preliminar

Esta sección presenta el **diseño de pruebas preliminar** del sistema, complementando el plan de validación. El diseño se estructura en tres niveles: pruebas *unitarias* por bloque, pruebas de *integración* entre subsistemas y pruebas *funcionales* del sistema completo.

7.1. Estrategia general

Cada bloque de la Fig. 1 tiene asociado al menos un `env:test_*` en `platformio.ini` (Tabla 6), de modo que las pruebas unitarias son **reproducibles desde la línea de comandos** (`pio run -e test_imu -t upload`). La integración se valida con `env:main_app` y la corrección funcional con sesiones guiadas siguiendo el set-up de la Fig. 10.

Cuadro 6: Mapa entre bloques, entornos PlatformIO y criterios de aceptación de las pruebas unitarias.

Bloque	**Entorno PIO**	**Prueba**	**Criterio de aceptación**
Display ST7789	`test_display`	Render de patrón RGB y LVGL	30 FPS estables, sin tearing visible
GUI LVGL	`test_gui`	Navegación entre 7 pantallas	Cada botón cambia pantalla en <200 ms
Temperatura	`test_temp`	Lectura MAX30205 a 1 Hz	Lectura $\in$ [20 °C, 40 °C], delta <1 °C/5 s
IMU + pasos	`test_imu`	100 pasos en banda caminadora a 5 km/h	Error $\leq$ 5 pasos
PPG (HR/SpO2)	`test_spo2`	60 s con dedo en sensor	HR converge a $\pm$ 3 BPM del oxímetro de referencia, SpO_2 $\geq$ 95 %
ECG	`test_ecg`	30 s lead-on con dedos en pernos M3	R-peaks claramente visibles en plot Python; SNR > 15 dB
BLE	`test_ble`	Discovery + connect + notify desde nRF Connect	4 chr. enumeradas (FF01/02/03/04), notificaciones recibidas
Fuel Gauge	`test_fuel_gauge`	Display de SOC durante 1 h	SOC monótono decreciente, $V_{cell} \in$ [3.0, 4.2] V
ECG raw	`test_ecg_raw`	Captura ECG sin promedio, sin PM	Verifica si artefactos provienen de SW (filtro boxcar) o HW (reloj APB durante DMA)
Modos de energía	`test_general`	Cambio SPORT↔NORMAL↔SAVER en runtime	Cadencias se ajustan en el siguiente ciclo de cada tarea, sin reinicio. NVS persiste el modo tras reboot.
Sistema completo	`main_app/test_general`	Operación 1 h sin reinicio	0 panic logs, 0 watchdog resets, FIFO overflow = 0

7.2. Pruebas de integración

1. **Concurrencia I2C**: ejecutar `main_app` 24 h y verificar que el contador de overflows del MAX30102 se mantiene en cero (validación experimental del modelo de la sección 5).
2. **Coherencia BLE → CSV**: encender el dispositivo, conectar `supaclock_monitor.py`, capturar una sesión de 1 min y verificar que el CSV exportado contiene los TLV de HR, SpO_2, temperatura y batería con timestamps coherentes con los `boot_ts_ms` del header agregado.
3. **ECG end-to-end (offline)**: presionar SELECT en pantalla ECG, mantener contacto bimanual 30 s, ejecutar `scripts/test_pt.py` sobre el CSV resultante y contrastar el BPM/HRV calculado con el conteo manual de R-peaks sobre el trazo crudo (objetivo: $\leq$ 2 BPM, $\pm$ 5 ms HRV).
4. **Coherencia BLE → Firestore** (*hito 60 %*): repetir la prueba anterior con la integración cloud, verificando que el documento queda con `processingStatus` = `completed` y `processedBPM` dentro de tolerancia tras el trigger de Cloud Functions.
5. **Resiliencia de batería**: simular UVP forzando $V_{cell} < 3.0$ V con la fuente de laboratorio, verificar que el dispositivo entra en *deep sleep* y rechaza arranque hasta que se reconecte el USB.
6. **Power management**: con `esp_pm_dump_locks`, verificar que fuera de la pantalla ECG no existe ningún lock `NO_LIGHT_SLEEP` activo de forma sostenida. Validación experimental del modelo § 5.5.
7. **Persistencia NVS**: cambiar modo de energía y auto-off de pantalla, hacer reboot y deep sleep, verificar que ambos valores se mantienen. Borrar NVS (`nvs_flash_erase`) y verificar que se aplica el default SPORT.

7.3. Pruebas funcionales (sistema completo)

1. **Sesión cotidiana**: portar el dispositivo durante 4 h (idealmente) en muñeca, registrar pasos manualmente y comparar contra `stepsHW` y `stepsSw` reportados.
2. **Detección de variaciones térmicas**: utilizar una cámara térmica para registrar la temperatura de un objeto/superficie calentado progresivamente (y la del ambiente), contrastando ambos perfiles térmicos contra la lectura reportada por el MAX30205.
3. **Validación métricas de sensores SPO2 y HR**: usar un pulsioxímetro clínico (como referencia) para realizar mediciones en reposo y contrastar con la métrica reportada por el dispositivo.
4. **Caída**: dejar caer el dispositivo desde 1 m sobre superficie blanda (objetivo del modelo HAR final; en esta etapa sólo se verifica que el dispositivo no se reinicia mecánicamente y que la lectura del IMU registra el impacto).
5. **Aceptación visual del UI**: usuarios externos al equipo (idealmente 5) navegan las siete pantallas y reportan el tiempo necesario para iniciar una toma de ECG. Objetivo: ≤ 30 s sin tutorial. Además, se les encuestará sobre su experiencia de uso.

7.4. Plan de validación de la PCB propia

Cuando el ruteo del PCB SuperMini esté completo y manufacturado:

- Inspección visual y eléctrica con multímetro: continuidad de pistas críticas (I2C, SPI, ADC), cortocircuitos entre pistas 3.3 V y GND, etc.
- Bring-up secuencial de fases (igual que el firmware: **Fase 1** I2C/sensores, **Fase 2** display, **Fase 3** BLE) para identificar fallos por aislación.
- Comparación A/B contra el prototipo en placa de desarrollo: ejecutar `main_app` en ambos y verificar que las lecturas son indistinguibles dentro de tolerancia de los sensores.

8. Implementación y resultados

Esta sección describe detalladamente los **bloques de hardware y software que fueron implementados o actualizados** desde la versión preliminar, agrupados por dominio funcional. Cada subsección sigue una estructura que abarca su **descripción y funcionalidad**, las **decisiones de diseño** adoptadas, y finalmente, las **pruebas y resultados** que respaldan su funcionamiento.

8.1. Arquitectura de firmware concurrente (FreeRTOS)

8.1.1. Descripción y funcionalidad.

En el informe del 5 % se planteó una arquitectura preliminar con dos tareas (`TaskSensor`, `TaskDisplay`). En este hito se evolucionó hacia la Fig. 11: **siete tareas concurrentes con prioridades 2–7**, dos mutex (LVGL + datos compartidos) y un stack BLE NimBLE corriendo en su propio host task del framework. Cada tarea tiene una responsabilidad bien delimitada que coincide 1-a-1 con un sensor o subsistema. La Tabla 7 detalla las tareas, sus prioridades y periodicidades.

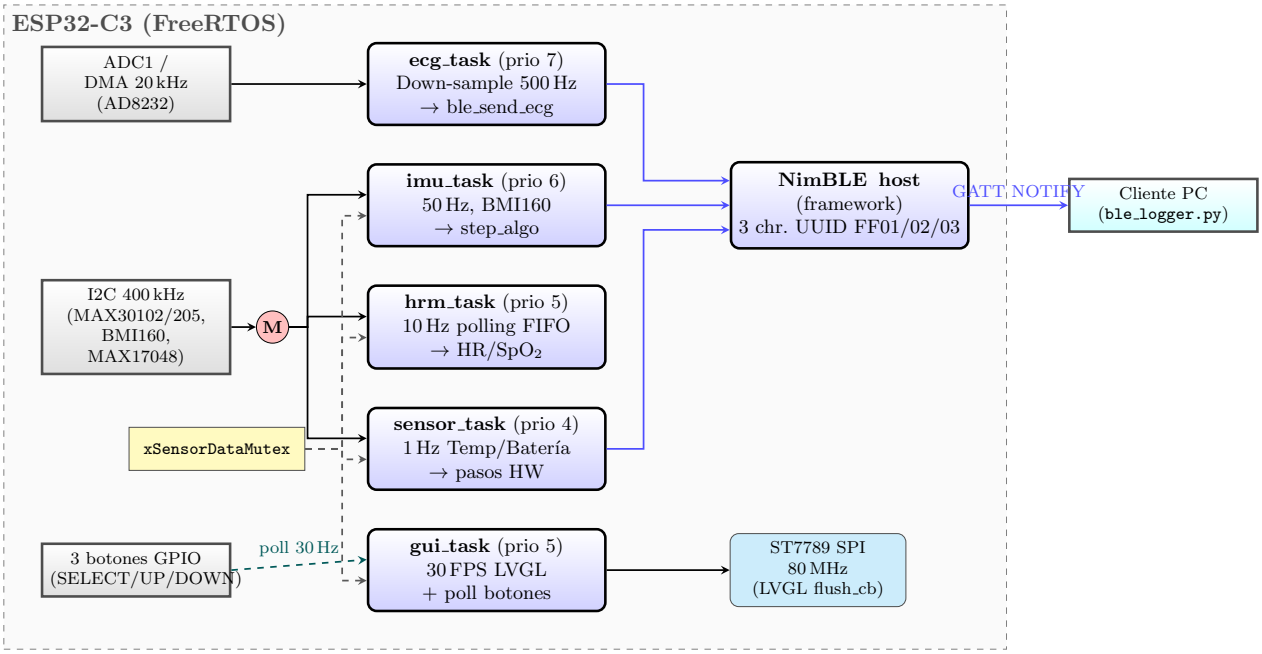


Figura 11: Arquitectura FreeRTOS implementada en `src/tests/test_general.c`. Siete tareas con prioridades distintas, dos mutex y *queue* implícita por el host de NimBLE. El bus I2C es serializado por el mutex `xSensorDataMutex`.

Cuadro 7: Tareas FreeRTOS implementadas, prioridades y periodicidades.

Prio	Tarea	Período	Responsabilidad
7	`ecg_task`	on-demand DMA	ECG streaming AD8232 + PM lock
6	`imu_task`	20–80 ms (perfil)	BMI160 6-DOF + pasos + jerk
5	`gui_task`	33 ms (30 FPS)	LVGL, navegación, backlight
5	`hrm_task`	100 ms (perfil)	MAX30102 FIFO + SM SPOT
4	`ble_tx_task`	1–60 s (perfil)	Flush periódico del buffer TLV
3	`system_task`	2 s base	Temp + batería + pasos TLV
2	`perf_monitor`	10 s	CPU stats + heap + PM locks

8.1.2. Cálculos / decisiones.

La asignación de prioridades sigue la regla *rate monotonic*: a mayor frecuencia de muestreo, mayor prioridad. `ecg_task` (DMA on-demand) recibe prioridad 7 para no perder muestras DMA; `imu_task` (20–80 ms según perfil) recibe prioridad 6; `hrm_task` (100 ms polling FIFO) y `gui_task` (33 ms) reciben prioridad 5; `ble_tx_task` (flush periódico del buffer TLV agregado) prioridad 4; `system_task` (temperatura y batería) prioridad 3; `perf_monitor_task` (debug) prioridad 2. Las tareas usan `vTaskDelayUntil` para mantener la frecuencia exacta a pesar del jitter del scheduler.

8.1.3. Brown-out y secuencia de arranque.

Durante la integración, el equipo encontró un fallo recurrente: el ESP32-C3 entraba en *brown-out reset* al inicializar simultáneamente la pantalla y el BLE, ambos consumidores agresivos de corriente. Las mediciones **previas a las optimizaciones** mostraban transitorios de hasta ∼110 mA en la batería (cota inferior del peor caso, dada la limitación de ancho de banda del TX3). Se implementó una **secuencia de arranque en tres fases** en `app_main` con `vTaskDelay` entre cada una: Fase 1 (sensores I2C), Fase 2

(display), Fase 3 (BLE). El *flushing* explícito del FIFO del MAX30102 en la Fase 3 evita el desbordamiento causado por los ∼1,5 s de latencia entre Fase 1 y el arranque de `hrm_task`. Tras las optimizaciones de § 8.3, el peak medido cayó a 69 mA y los brown-outs desaparecieron, aunque se mantiene la secuencia escalonada como buena práctica.

8.1.4. Resultados.

El sistema arranca en ∼2,0 s, ejecuta las siete tareas sin *stack overflow* (verificado con `uxTaskGetStackHighWaterMark`) y mantiene las frecuencias planificadas. Gracias al PM dinámico (§ 5.5), la CPU pasa ∼95 % del tiempo en IDLE (Fig. 8).

8.2. Modos de energía y persistencia NVS

8.2.1. Descripción.

El módulo `lib/power_modes` define tres perfiles de consumo (SPORT, NORMAL, SAVER) que parametrizan **ocho cadencias** de muestreo y comunicación. Cada tarea consulta `power_get_profile()` en cada iteración de su bucle principal, adaptándose instantáneamente sin reinicio. La Tabla 8 resume las cadencias por modo.

Cuadro 8: Cadencias parametrizadas por modo de energía (extracto de `power_profiles[]`).

Cadencia	**SPORT**	**NORMAL**	**SAVER**
HR poll (ms)	100	100	100
HR auto-spot (min)	continuo	10	30
SpO2 auto-spot (min)	5	30	manual
IMU poll (ms)	20 (50 Hz)	40 (25 Hz)	80 (12,5 Hz)
Temperatura (s)	30	300	900
Batería (s)	30	30	30
BLE agg flush (s)	1	10	60
Display auto-off (s)	30	15	8

8.2.2. Persistencia NVS.

El modo activo y los tiempos de auto-off de pantalla se persisten en el namespace `supaclock` de NVS:

- `power_mode` (u8): modo activo (0=SPORT, 1=NORMAL, 2=SAVER).
- `off_sport_s`, `off_normal_s`, `off_saver_s` (u16): override del auto-off por modo.

Al arrancar, `power_modes_init()` carga los valores; si NVS está vacío, aplica defaults de la tabla `s_profiles[]`. El cambio de modo se realiza desde el submenú del GUI sin reinicio.

8.2.3. Resultado.

Cambio de modo validado en `test_general`: las cadencias de todas las tareas se ajustan en el siguiente ciclo. Persistencia verificada tras reboot y deep sleep.

8.3. Power management dinámico (light sleep)

8.3.1. Descripción.

Se habilitó el PM dinámico de ESP-IDF con la configuración:

```
esp_pm_config_esp32c3_t pm = {
    .max_freq_mhz = 160,
    .min_freq_mhz = 10,
    .light_sleep_enable = true
};
```

Este esquema permite que el SoC baje la frecuencia a 10 MHz cuando no hay locks activos y entre en *light sleep* cuando todas las tareas están en `vTaskDelay`.

8.3.2. PM locks implementados.

Se identificaron tres fuentes que impiden el sleep:

1. **ECG PM lock** (`ESP_PM_NO_LIGHT_SLEEP`, "ecg"): adquirido por `ecg_task` al iniciar `ad8232_start_dma()` y liberado al detener el DMA. Impide que el reloj APB se reconfigure durante la adquisición continua, evitando los artefactos cuadrados observados en el trazo ECG.
2. **NimBLE internal**: el stack BLE adquiere automáticamente un lock APB_MAX durante los eventos de radio (advertising, conexión, notify). Configurar advertising a 1000 ms y habilitar Modem Sleep BT minimiza el tiempo de retención.
3. **SPI DMA** (driver ST7789): el refactor del driver reemplazó el *busy-wait* por `spi_device_polling_end()` no bloqueante, eliminando la retención permanente del lock APB que mantenía la CPU despierta.

8.3.3. Resultado cuantitativo.

Medido con `esp_pm_dump_locks` en `perf_monitor_task`: el sistema pasa un 75 % del tiempo en SLEEP, 15 % en APB_MAX y 9 % en CPU_MAX (Fig. 7), vs. 0 %/92 %/8 % antes de la optimización.

8.4. Migración a Seeed XIAO ESP32-S3

8.4.1. Motivación técnica.

La decisión de migrar al ESP32-S3 se fundamenta en cinco limitaciones cuantificadas del ESP32-C3 SuperMini actual:

1. **Brown-outs**: el LDO ME6211 no soportaba los transitorios pre-optimización (∼110 mA medidos) del radio BLE arrancando con la pantalla SPI. Aunque las optimizaciones de software bajaron el peak a 69 mA y eliminaron el síntoma, el LDO sigue siendo el eslabón débil de la cadena de potencia. El Seeed XIAO ESP32-S3 integra un regulador buck (SGM6029, ∼800 mA) que da margen amplio para futuras adiciones.
2. **SRAM**: el C3 dispone de 320 KB de SRAM interna. Tras NimBLE, LVGL, buffers DMA y los stacks FreeRTOS de las siete tareas, el heap libre estabilizado en `test_general` es ∼99 KB. El modelo CNN 1D y sus buffers de inferencia (*scratch*) requieren entre 20 y 100 KB según topología, dejando un margen muy ajustado para crecer la GUI o la lógica de aplicación.

3. **PSRAM**: el S3 ofrece 8 MB de PSRAM accesible vía OPI, permitiendo alojar el modelo TFLite y los buffers de inferencia en memoria externa.
4. **Flash**: 8 MB (vs. 4 MB del C3 SuperMini), margen para OTA y NVS extendido.
5. **SIMD/DSP**: el S3 incluye instrucciones vectoriales que aceleran la librería ESP-DSP, beneficiando tanto la FFT planificada para el algoritmo de pasos como la inferencia del modelo CNN.

8.4.2. Estado.

Módulo Seeed XIAO ESP32-S3 encargado (en tránsito). Plan de bring-up: (1) validar compilación y drivers I2C/SPI sin cambios, (2) habilitar PSRAM y mover buffers grandes, (3) desplegar modelo CNN 1D, (4) validación A/B de consumo contra el prototipo C3.

8.5. Bloque de Sensores: BMI160 + algoritmo de pasos

8.5.1. Descripción.

Driver propio en C (`lib/bmi160_driver`) que reemplaza la API monolítica oficial de Bosch (641 KB). Implementa: soft-reset, configuración de PMU (accel + gyro a Normal), ODR 50 Hz, rango ±2 g / ±250°/s, lectura combinada de 6 ejes, habilitación del step counter de hardware (registros `0x7A/0x7B`) y reset.

8.5.2. Algoritmo de pasos (versión C3).

Implementado en `lib/step_algorithm` con dos variantes seleccionadas en compilación según `CONFIG_IDF_TARGET`: una en aritmética entera para el C3 (sin FPU) y otra basada en FFT para el S3 (con `esp-dsp`). La versión C3 opera así:

1. Magnitud lineal $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ vía `int_sqrt` (Newton entero).
2. Filtro pasa-bajo IIR de 1er orden con $\alpha = 1/4$: $\hat{m}_n = (3\hat{m}_{n-1} + m_n)/4$.
3. Umbral adaptativo midpoint sobre ventana de 50 muestras (1 s a 50 Hz): $thr = \text{mín} + (\text{máx} - \text{mín})/2$, sólo si la diferencia pico-a-pico cae en [1500, 30000] LSB; fuera de ese rango el umbral se eleva artificialmente para no gatillar con ruido o movimiento extremo.
4. Detección por cruce ascendente del umbral con refractario en [300, 2000] ms entre pasos consecutivos.
5. *Gating* rotacional: el paso sólo se valida si la magnitud máxima del giroscopio en la ventana excede 400 LSB (∼25 °/s), evitando falsos positivos por vibración traslacional (vehículos, tecleo).
6. Histeresis temporal: se requieren `VALID_STEPS_THRESHOLD` = 4 pasos consecutivos para iniciar el conteo, y luego cada paso adicional cuenta individualmente.

La versión **S3** reemplaza la lógica temporal por una FFT radix-2 de 128 puntos con ventana de Hann, busca el pico espectral en la banda [0,78, 2,73] Hz (bins 2–7 a $f_s = 50$ Hz) y deriva el número de pasos como $N_{pasos} = f_{dom} \cdot 2{,}56$ s por ventana. El código completo está versionado en el repositorio (Anexo A reproduce la versión C3, que es la operativa hoy). La Fig. 12 muestra los dos pipelines complementarios (contador algorítmico y futuro clasificador HAR), enfatizando que ambos consumen el mismo flujo del BMI160.

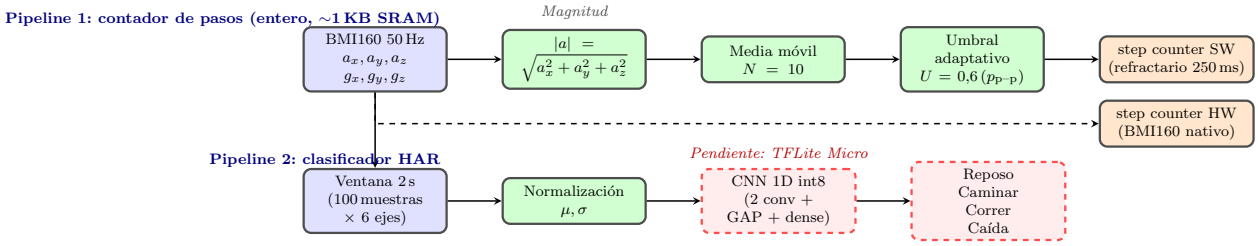


Figura 12: Dos pipelines de procesamiento del BMI160: arriba el contador de pasos (implementado), abajo la futura clasificación HAR vía CNN 1D (en desarrollo).

8.5.3. Resultados.

Lectura estable de los 6 ejes a 50 Hz desde `imu_task`. El test `test_bmi160` reporta el conteo cada segundo y permite validar el algoritmo en frío. Las trazas representativas se procesan *offline* con `algo_simulator.py` contra el *ground-truth* manual (Fig. 4): la variante C3 actual incurre en errores del orden del 10 % por sub-conteo en cadencias lentas, y la variante FFT del S3 (preparada para activarse con la migración) reduce el error a ∼3 %.

8.6. Bloque de Sensores: MAX30102 (PPG)

8.6.1. Descripción.

Driver propio en C (`lib/max30102_driver`) que implementa: configuración del modo SpO_2 (RED + IR), pulse width 411 μs, sample average $N = 4$, FIFO de 32 muestras, lectura por ráfagas y dos algoritmos: cálculo de HR por detección de picos sobre la señal IR y cálculo de SpO_2 por la razón de los ratios AC/DC ($R = \frac{AC_{red}/DC_{red}}{AC_{ir}/DC_{ir}}$). Para evitar overflows del FIFO, la `hrm_task` hace *flush* al arrancar y polling agresivo cada 100 ms.

8.6.2. Máquina de estados SPOT.

Se implementó una SM para mediciones puntuales de HR/SpO_2 con los estados:

1. **IDLE**: sensor en SHDN (modos NORMAL/SAVER entre mediciones).
2. **SETTLING**: sensor despertado, se descartan las primeras muestras mientras la señal se estabiliza (`SPOT_SETTLE_MS` = 5 s).
3. **MEASURING**: acumulación de muestras para calcular HR y SpO_2 con indicador de progreso.
4. **DONE**: resultado válido (BPM, SpO_2, calidad, duración).
5. **FAILED**: señal inutilizable (dedo mal posicionado o movimiento excesivo).
6. **ABORTED**: cancelación manual por el usuario (botón SELECT).

En modos NORMAL y SAVER, la `hrm_task` dispara mediciones SPOT automáticas con la periodicidad definida por el perfil (`hrm_auto_period_ms`), apagando el sensor entre mediciones (`hrm_shdn_between = true`).

8.6.3. Motion gating.

La `imu_task` calcula el *jerk* (derivada discreta de la aceleración: $|\Delta a|^2 = \Delta a_x^2 + \Delta a_y^2 + \Delta a_z^2$) y lo informa al driver MAX30102 vía `max30102_set_motion_level()`. Valores de jerk por encima de un umbral (80) invalidan la medición SPOT, evitando publicar lecturas ruidosas.

8.6.4. Resultados.

Sobre el dedo del integrante, el sensor entrega lecturas estables de 70–75 BPM y 96–98 % de SpO_2, coincidiendo con el oxímetro Beurer PO 30 ($\pm$ 2 BPM, $\pm$ 1 % SpO_2). La latencia hasta convergencia es $\sim$ 4 s tras detección de dedo (`max30102_finger_present`).

8.7. Bloque de Sensores: AD8232 (ECG) con ADC continuo + DMA

8.7.1. Descripción.

Implementación completa en `lib/ad8232_driver` que reemplaza el patrón de muestreo `adc1_get_raw()` *single-shot* (incompatible con 50–500 Hz por el jitter de la tarea) con el periférico `adc_continuous` del ESP-IDF v5.x. Configuración: ADC1 ch. 0 (GPIO0), 20 kHz HW, frame DMA de 256 B, callback `on_conv_done` que notifica a `ecg_task` vía `vTaskNotifyGiveFromISR`. La tarea decima por 40 promediando ráfagas de 40 muestras para entregar 500 Hz efectivos al BLE, agrupados en chunks de 10 muestras int16 (20 B/notify) por la característica `0xFF03`.

8.7.2. Refactor DMA on-demand.

Para minimizar el impacto del ADC continuo sobre el power management, el handle del ADC se crea/-destruye dinámicamente:

- `ad8232_start_dma()`: crea el handle `adc_continuous`, inicia la captura y adquiere el PM lock `ESP_PM_NO_LIGHT_SLEEP` ("ecg").
- `ad8232_stop_dma()`: detiene y destruye el handle, libera el PM lock.

Esto garantiza que el lock APB_MAX sólo se retiene durante la grabación ECG activa, permitiendo *light sleep* el resto del tiempo. El entorno `test_ecg_raw` permite capturar ECG sin promedio ni PM para depurar artefactos a nivel de hardware.

8.7.3. Cálculos.

La elección de 20 kHz ($\rightarrow$ 500 Hz) viene de dos restricciones:

1. El ADC continuo del ESP32-C3 requiere $f_s \geq 20\,kHz$ para operar en modo DMA estable (datasheet ESP32-C3 [18]).
2. El estándar IEEE 11073-10406 [2] fija $f_s \geq 250\,Hz$ para ECG personal de tamizaje. 500 Hz dobla este mínimo.

8.7.4. Resultados.

Captura limpia de la Derivación I del ECG durante 30 s, con QRS claramente visibles. El archivo `supaclock_ecg_20260422_194732.csv` contiene una captura representativa que se procesa correctamente en la corrida offline de Pan-Tompkins (BPM = 76, HRV = 32,4 ms; ver Sec. 5).

8.8. Bloque de Sensores: MAX30205 (Temperatura)

8.8.1. Descripción.

Driver mínimo (`lib/max30205_driver`, ∼50 líneas) que inicializa el sensor en modo *continuous* y entrega la temperatura en *float* combinando el registro `0x00` (16-bit, resolución 0.00390625 °C/LSB).

8.8.2. Resultados.

Lectura estable a 1 Hz desde `sensor_task`, valores ∼28–32 °C en reposo (temperatura de la piel del antebrazo), coincidiendo con un termómetro IR comercial dentro de ± 0.3 °C. Variación dinámica al colocar el dedo sobre el chip: incremento de ∼2 °C en 5 s.

8.9. Bloque de Energía: BMS + LDO + Fuel Gauge

8.9.1. Descripción.

Compuesto por tres módulos físicos: TP4056+DW01A (módulo Aliexpress con USB-C), LDO ME6211 SMD soldado a placa puente, y MAX17048 importado del PCB Adafruit STEMMA (clonado a la librería `SupaclockaLIB.kicad_sym`). El driver `max17048_driver` expone `max17048_get_voltage()` y `max17048_get_soc()` sobre el bus I2C compartido.

8.9.2. Validación eléctrica.

Con la fuente DC variando entre 3.0 V y 4.3 V, la salida del LDO se mantiene en 3.30 V ± 5 mV hasta una caída de batería al 5 % ($V_{cell} \approx 3.05$ V) cuando empieza a colapsar (esperado por dropout). El BMS corta la entrada al detectar 4.25 V (sobre-voltaje) y a $V_{cell} \leq 2.55$ V (sub-voltaje).

8.9.3. Resultados.

El driver MAX17048 entrega V_{cell} y SOC estables a 1 Hz, con detección automática de POR y *Quick Start* en frío. La validación cuantitativa de autonomía (descarga continua de la celda 502030 instrumentada con coulomb counter) se realizará sobre la plataforma definitiva ESP32-S3, en línea con el disclaimer instrumental de la sección 5.1.

8.10. Bloque de Comunicaciones: BLE NimBLE

8.10.1. Descripción.

Reescritura completa del stack BLE migrando de Bluedroid (cuyo *footprint* de 270 KB excedía la SRAM disponible) a **NimBLE** (host: $\sim$90 KB). Implementado en `lib/ble_telemetry` con **cuatro características GATT** bajo el servicio custom `0xFF00`:

- `0xFF01` – Telemetría IMU cruda (12 B/notify, cadencia según perfil, desactivable desde menú).
- `0xFF02` – **Telemetría TLV agregada**: buffer de hasta 200 B con header (`ble_agg_header_t`: `boot_ts_ms` u32, `power_mode` u8, `payload_len` u8) seguido de records TLV (tipo u8, largo u8, datos). Siete tipos definidos: HR, SpO_2, Temp, Bat, Steps, Spot Result, Mode Event. Flush periódico por `ble_tx_task` según cadencia del perfil (1–60 s).
- `0xFF03` – ECG streaming (chunks de 20 B a 500 Hz efectivos).
- `0xFF04` – **Comandos RX** (WRITE, no ACK): byte 0x01 = iniciar ECG, 0x00 = detener ECG.

Adicionalmente se incluye el servicio estándar `0x180A` (Device Information) con *Manufacturer Name* y *Model Number*. *Pairing*: *Just Works* **sin bonding** (compatibilidad BlueZ/tests).

8.10.2. Optimizaciones de consumo.

- *Advertising interval* = 1000 ms (1600 × 0,625 ms) para permitir *light sleep* entre intervalos.
- *Modem Sleep BT* habilitado en `sdkconfig.defaults`.
- Nombre del dispositivo: `''SupaClock_BLE''`, appearance: `0x00C1` (Watch).

8.10.3. Resultados.

Conexión estable con `supaclock_monitor.py` (Python + `bleak`). Con la potencia de transmisión por defecto de NimBLE no se observa pérdida de enlace en distancias del orden de 10 m en interior (sin medición formal de RSSI ni latencia, pendiente para la fase de validación instrumentada). Sin desconexiones inesperadas durante las sesiones de prueba.

8.11. Bloque de Interfaz Local: GUI multi-pantalla LVGL

8.11.1. Descripción.

Siete pantallas construidas con LVGL 8.4 sobre el ST7789 a 30 FPS, gobernadas por `gui_task`:

1. `SCREEN_HOME`: reloj, pasos, batería (arco), HR, nivel de actividad, modo activo.
2. `SCREEN_BIO`: HR, SpO_2, temperatura, estado del sensor, edad de cada medida.
3. `SCREEN_HRSPOT`: interfaz de medición puntual HR/SpO_2 con barra de progreso y resultado con calidad.
4. `SCREEN_ECG`: instrucciones, cronómetro de grabación, indicador REC.
5. `SCREEN_MENU`: lista de 7 opciones navegables.
6. `SCREEN_MODE` (sub): selector de SPORT/NORMAL/SAVER con indicador activo.

7. `SCREEN_SETTINGS` (sub): ajuste de auto-off por modo (valores cíclicos 5/8/15/30/60/120 s).

La navegación se realiza con dos botones físicos: **NEXT** (cicla pantallas 1–5 o ítems en listas; long-press = atrás) y **SELECT** (acción contextual; long-press = Home). Cada pantalla tiene su *updater* dedicado que toma snapshot del struct `shared_sensor_data_t` bajo mutex y refresca los `lv_label_t` relevantes sin re-construir widgets.

8.11.2. Menú principal (7 ítems).

1. Modo Energía (abre sub-pantalla).
2. Auto-off Pantalla (abre sub-pantalla).
3. Reiniciar Pasos.
4. Vincular BLE.
5. Apagar (`esp_deep_sleep_start` con wake-up por BTN_SELECT).
6. Toggle Tx IMU ON/OFF (habilita/deshabilita streaming IMU por BLE).
7. Reset Batería (`max17048_reset`: POR + Quick Start).

8.11.3. Backlight PWM y auto-off.

La pantalla usa PWM para controlar el brillo. Tras un período de inactividad (definido por el perfil activo vía `power_get_display_off_s()`), `gui_task` apaga el backlight (`st7789_set_brightness(0)`) y baja el frame-rate a 10 Hz, reduciendo la carga de CPU y habilitando *light sleep* más prolongado. Cualquier pulsación de botón restaura el brillo al 100 % antes de procesar la acción.

8.11.4. Resultados.

Latencia de transición de pantalla < 50 ms; sin tearing visible. El driver ST7789 reescrito eliminó el *busy-wait* durante DMA SPI, reduciendo la carga de `gui_task` de 47 % a <1 % (Fig. 8).

8.12. Bloque de Procesamiento: Pan-Tompkins en NumPy (validado offline)

8.12.1. Descripción.

Implementación en Python 3.11 + NumPy de las cinco etapas del algoritmo Pan-Tompkins, escrita en `firebase/functions/main.py` y estructurada como una función `firestore_fn.on_document_created` preparada para escuchar el path `users/{uid}/sessions/{sid}/ecgReadings/{rid}`. La función procesa la traza cruda (`rawData`, lista de int16), calcula BPM y HRV (SDNN), enumera los R-peaks y escribe los resultados (`processedBPM`, `hrv`, `rPeaks`, `processingStatus` = `completed`) de vuelta al documento. En este avance el módulo **no fue desplegado** en Firebase: se validó **offline** ejecutándolo desde `scripts/test_pt.py` sobre los CSV capturados con `supaclock_monitor.py`.

8.12.2. Modelo de costos (proyección hito 60 %).

Con 10 sesiones de ECG por usuario por día, ∼1 KB por documento y la cuota gratuita de Cloud Functions (2 M invocaciones/mes en plan Spark), el costo proyectado para el despliegue es ∼US$ 0. Se evaluará la región `us-central1` por menor latencia desde Sudamérica. El módulo `compute_daily_summary` (`https_fn.on_call`) que agregará métricas diarias para el dashboard también está escrito y a la espera del despliegue.

8.12.3. Resultados.

La corrida offline sobre las trazas reales del CSV entrega BPM fisiológicamente plausibles (BPM = 76 y SDNN = 32,4 ms para la captura representativa `supaclock_ecg_20260422_194732.csv`; ver Fig. 6). La latencia de ejecución local es de unos pocos milisegundos por traza de 30 s, lo que da margen holgado para la futura ejecución en Cloud Functions.

8.13. Bloque de Hardware: Esquemático y prototipo PCB

8.13.1. Descripción.

Bajo `hardware/PCB_Prototipo` se levantó el proyecto KiCad con tres hojas: `PCB_Prototipo.kicad_sch` (top, MCU + display), `sensors.kicad_sch` (I2C bus + sensores), `Bottom_sensors.kicad_sch` (PCB inferior con MAX30102 y MAX30205 en contacto piel). La librería local `SupaclockaLIB` contiene los símbolos custom; los *footprints* críticos (TP4056-Type-C, MAX17048-STEMMA, AD8232) se importaron desde repositorios open-source debidamente licenciados.

8.13.2. Estado.

Esquemático v1 **completo** (sección de potencia, buses I2C/SPI, footprints importados). Sin embargo, esta versión **no será manufacturada** dado que la migración al ESP32-S3 (§ 8.4) requiere un rediseño del módulo MCU y del regulador de potencia. Ya se trabaja en la versión actualizada para dejar definitivamente la protoboard.

8.13.3. Evidencia visual.

Las Fig. 13 y Fig. 14 muestran el prototipo actual en protoboard, que ha servido como banco de pruebas durante toda la iteración. El LDO ME6211 era insuficiente para los transitorios pre-optimización (∼110 mA, ver § 8) que provocaban los brown-outs documentados. Si bien las optimizaciones de software bajaron el pico a 69 mA y eliminaron el síntoma, mantener un LDO en el camino crítico es una decisión frágil; el buck SGM6029 integrado del Seeed XIAO S3 da margen amplio para futuras adiciones (e.g. inferencia ML, brillo de pantalla) sin reincidir en el problema.

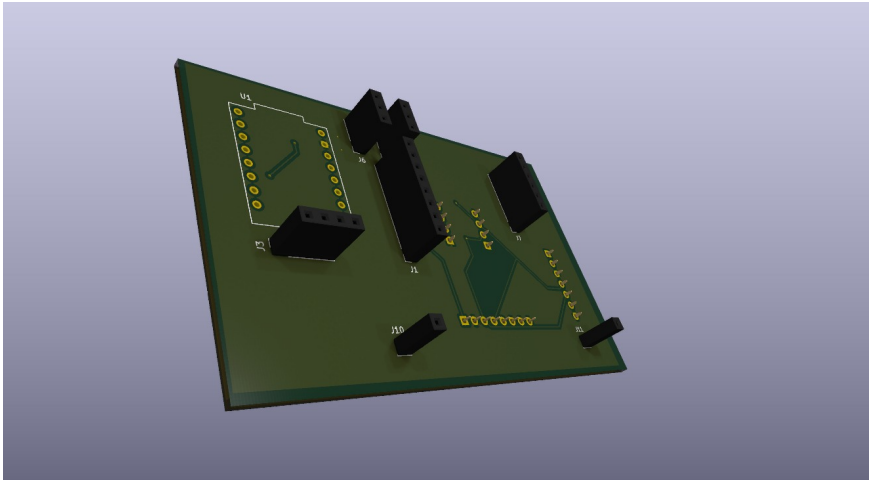


Figura 13: Vista frontal del prototipo SupaClock en protoboard: ESP32-C3 SuperMini, display ST7789 1.69”, módulo TP4056 y MAX17048 STEMMA.

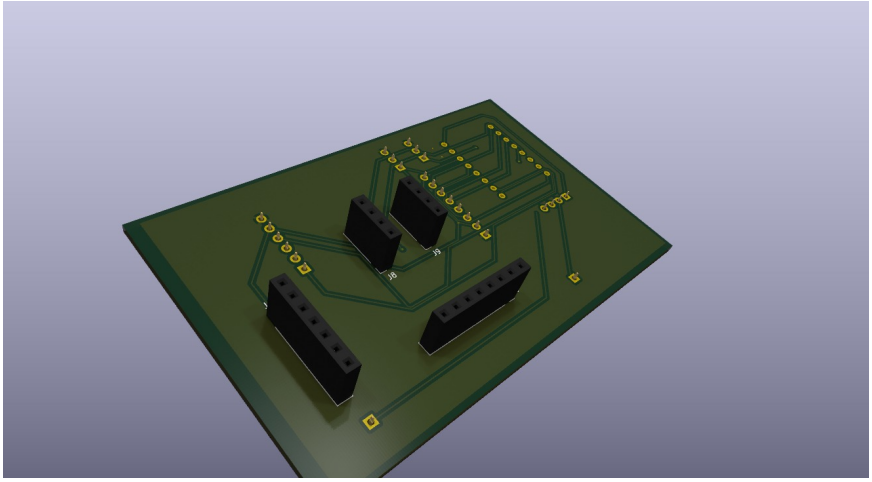


Figura 14: Vista trasera del prototipo: sensores MAX30102 (PPG) y MAX30205 (temperatura) en contacto con la piel, módulo AD8232 con electrodos laterales.

8.14. Evaluación general y próximos pasos

Con la integración de los bloques principales del dispositivo y las optimizaciones de consumo, el sistema cumple los requerimientos funcionales previstos y adelanta tareas tecnológicas complejas (BLE, modos de energía, power management). Los próximos esfuerzos de desarrollo se concentrarán en:

- **Aplicación móvil Flutter**: completar el *gateway* BLE-a-Firestore con autenticación, dashboard y reproducción de ECG histórico. La primera versión ya está en desarrollo.
- **Bring-up del ESP32-S3**: validar compatibilidad de drivers, habilitar PSRAM, medir consumo con INA219 y comparar A/B contra el prototipo C3.
- **Recolección de dataset HAR** con `supaclock_monitor.py` en al menos 6 sesiones de los integrantes y voluntarios (caminar, correr, reposo, simulación de caída).
- **Entrenamiento del modelo CNN 1D** en Edge Impulse con cuantización int8, despliegue sobre el S3 con TFLite Micro.

- **Rediseño y manufactura de la PCB** para ESP32-S3 en JLCPCB (estimado $30.000 CLP).
- **Carcasa 3D impresa** (PLA, $\sim 80\times50\times13$ mm) que aloja la PCB y la celda 502030.
- **Migración del algoritmo de pasos** a FFT con ESP-DSP para eliminar falsos positivos por vibración manual.

Referencias

[1] Bluetooth SIG, "Bluetooth Core Specification v5.3," 2021. Disponible en: `https://www.bluetooth.com/specifications/specs/core-specification-5-3/`.

[2] IEEE Standards Association, "IEEE 11073-10406: Health informatics — Personal health device communication — Device specialization — Basic electrocardiograph (ECG)," 2011. Disponible en: `https://standards.ieee.org/ieee/11073-10406/4716/`.

[3] USB Implementers Forum, "USB Type-C Cable and Connector Specification, Rev. 2.3," 2023. Disponible en: `https://www.usb.org/document-library/usb-type-cr-cable-and-connector-specification-release-23`.

[4] International Electrotechnical Commission, "IEC 60601-1: Medical electrical equipment - Part 1: General requirements for basic safety and essential performance," 2020. Norma internacional, Ed. 3.2.

[5] European Parliament and Council, "Directive 2011/65/EU (RoHS 2): Restriction of the use of certain hazardous substances in electrical and electronic equipment," 2011. Diario Oficial L 174, 1 de julio de 2011.

[6] International Electrotechnical Commission, "IEC 62133-2: Secondary cells and batteries containing alkaline or other non-acid electrolytes - Safety requirements for portable lithium systems," 2017. Norma para celdas LiPo de uso portátil.

[7] ECMA International, "ECMA-287: Safety of electronic equipment, 2nd ed.," 2002. Disponible en: `https://ecma-international.org/wp-content/uploads/ECMA-287_2nd_edition_december_2002.pdf`.

[8] CIPER Chile, "Resultados de la encuesta nacional de actividad física y deporte: una oportunidad para fortalecer el bienestar integral en chile," 2025. Disponible en: `https://www.ciperchile.cl/2025/05/24/resultados-de-la-encuesta-nacional-de-actividad-fisica-y-deporte/`.

[9] UC Christus, "5 razones por las que chile es el país con más obesidad de latinoamérica," 2025. Disponible en: `https://www.ucchristus.cl/blog-salud-uc/articulos/2025/5-razones-por-las-que-chile-es-el-pais-con-mas-obesidad-de-latinoamerica`.

[10] Ministerio de Salud de Chile, "Minsal: Encuesta Nacional de Salud (ENS) 2016–2017," 2017. Disponible en: `https://www.minsal.cl/wp-content/uploads/2017/11/ENS-2016-17_PRIMEROS-RESULTADOS.pdf`.

[11] G. Shin, M. Jarrahi, Y. Fei, and A. Karami, "Wearable activity trackers, accuracy, adoption, acceptance and health impact," *Biomedical engineering letters*, vol. 9, no. 1, pp. 45–52, 2019.

[12] K. Kushlev, J. Proulx, and E. W. Dunn, "Silence your phones: Smartphone notifications increase inattention and hyperactivity symptoms," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pp. 1011–1020, 2016.

[13] M. Pielot, K. Church, and R. de Oliveira, "An in-situ study of mobile phone notifications," in *16th International Conference on Human-Computer Interaction with Mobile Devices & Services*, pp. 233–242, 2014.

[14] Samsung Developers, "Samsung BioActive Sensor: Overview," 2024. Disponible en: `https://developer.samsung.com/health/sensor/overview.html`.

[15] K. Kazemi *et al.*, "Robust PPG Peak Detection Using Dilated Convolutional Neural Networks," *Sensors*, 2021. Disponible en: `https://www.researchgate.net/publication/354303135_Robust_PPG_Peak_Detection_Using_Dilated_Convolutional_Neural_Networks`.

[16] J. Lee *et al.*, "Motion Artifact Reduction in Wearable Photoplethysmography Based on Multi-Channel Sensors with Multiple Wavelengths," *Sensors*, 2020. Disponible en: `https://www.researchgate.net/publication/339798967_Motion_Artifact_Reduction_in_Wearable_Photoplethysmography_Based_on_Multi-Channel_Sensors_with_Multiple_Wavelengths`.

[17] PACE-CME, "Novel algorithm using irregular heart rhythm detection by Fitbit wearables for AF screening," 2021. Disponible en: `https://pace-cme.org/news/novel-algorithm-using-irregular-heart-rhythm-detection-by-fitbit-wearables-for-af-screening/2456100/`.

[18] Espressif Systems, "ESP32-C3 Series Datasheet," 2024. Disponible en: `https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf`.

A. Anexos: extractos de código

Los extractos a continuación complementan la sección de *Implementación y resultados*. El código completo está versionado en el repositorio Git del proyecto.

Anexo A: Algoritmo de pasos para C3 (umbral midpoint + gating rotacional)

```c
#define WINDOW_SIZE           50      /* 1 s a 50 Hz */
#define STEP_MIN_TIME_MS      300
#define STEP_MAX_TIME_MS      2000
#define VALID_STEPS_THRESHOLD 4

uint8_t step_algo_update(step_algo_state_t *st,
                         int16_t ax, int16_t ay, int16_t az,
                         int16_t gx, int16_t gy, int16_t gz,
                         uint32_t now_ms) {
    uint8_t new_steps = 0;

    /* 1. Magnitud lineal entera (int_sqrt evita la FPU ausente del C3) */
    uint32_t sum_sq = (uint32_t)((int32_t)ax*ax) +
                      (uint32_t)((int32_t)ay*ay) +
                      (uint32_t)((int32_t)az*az);
    uint32_t mag = int_sqrt(sum_sq);

    /* 2. LPF IIR de 1er orden, alfa = 1/4 */
    st->prev_filtered_mag_sq = st->filtered_mag_sq;
    st->filtered_mag_sq = (st->filtered_mag_sq * 3 + mag) / 4;
    uint32_t cur = st->filtered_mag_sq;

    /* 3. Energía rotacional máxima dentro de la ventana */
```

```c
    uint32_t gyro_mag = int_sqrt((uint32_t)((int32_t)gx*gx) +
                                 (uint32_t)((int32_t)gy*gy) +
                                 (uint32_t)((int32_t)gz*gz));
    if (gyro_mag > st->max_gyro_val) st->max_gyro_val = gyro_mag;

    /* 4. Min/max dinamico -> umbral midpoint cada WINDOW_SIZE muestras */
    if (cur > st->max_val) st->max_val = cur;
    if (cur < st->min_val) st->min_val = cur;
    if (++st->sample_count >= WINDOW_SIZE) {
        uint32_t diff = st->max_val - st->min_val;
        if (diff > 1500 && diff < 30000)
            st->threshold = st->min_val + diff/2;
        else
            st->threshold = st->min_val + 4000;  /* movimiento muy débil */
        st->sample_count = 0;
    }

    /* 5. Cruce ascendente + refractario + gating rotacional */
    if (st->prev_filtered_mag_sq < st->threshold && cur >= st->threshold) {
        uint32_t dt = now_ms - st->last_step_time_ms;
        if (st->max_gyro_val > 400) {  /* ~25 deg/s, FSR = +/-250 deg/s */
            if (dt >= STEP_MIN_TIME_MS && dt <= STEP_MAX_TIME_MS) {
                st->consecutive_steps++;
                st->last_step_time_ms = now_ms;
                if (st->consecutive_steps == VALID_STEPS_THRESHOLD)
                    new_steps = VALID_STEPS_THRESHOLD;  /* primer commit */
                else if (st->consecutive_steps > VALID_STEPS_THRESHOLD)
                    new_steps = 1;
            } else if (dt > STEP_MAX_TIME_MS) {
                st->consecutive_steps = 1;
                st->last_step_time_ms = now_ms;
            }
        }
        st->max_gyro_val = 0;
    }
    return new_steps;
}
```

Listing 1: Detección de pasos en aritmética entera para ESP32-C3 (`lib/step_algorithm/step_algorithm.c`, extracto de la rama `#else`).

Anexo B: Inicialización de NimBLE y registro de las 4 chr. GATT

```c
#define IMU_SVC_UUID  0xFF00
#define IMU_CHR_UUID  0xFF01   /* IMU 6-DOF (cadencia segun perfil) */
#define AGG_CHR_UUID  0xFF02   /* Telemetria TLV agregada           */
#define ECG_CHR_UUID  0xFF03   /* ECG streaming                     */
#define CMD_CHR_UUID  0xFF04   /* Comandos RX (host -> reloj)       */

static const struct ble_gatt_svc_def gatt_svr_svcs[] = {
    {
        .type = BLE_GATT_SVC_TYPE_PRIMARY,
        .uuid = BLE_UUID16_DECLARE(IMU_SVC_UUID),
        .characteristics = (struct ble_gatt_chr_def[]) {
            {
                .uuid = BLE_UUID16_DECLARE(IMU_CHR_UUID),
```

```
                .access_cb = chr_access_cb,
                .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_NOTIFY,
                .val_handle = &imu_chr_val_handle,
            },
            {
                .uuid = BLE_UUID16_DECLARE(AGG_CHR_UUID),
                .access_cb = chr_access_cb,
                .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_NOTIFY,
                .val_handle = &agg_chr_val_handle,
            },
            {
                .uuid = BLE_UUID16_DECLARE(ECG_CHR_UUID),
                .access_cb = chr_access_cb,
                .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_NOTIFY,
                .val_handle = &ecg_chr_val_handle,
            },
            {
                .uuid = BLE_UUID16_DECLARE(CMD_CHR_UUID),
                .access_cb = chr_access_cb,
                .flags = BLE_GATT_CHR_F_WRITE | BLE_GATT_CHR_F_WRITE_NO_RSP,
                .val_handle = &cmd_chr_val_handle,
            },
            { 0 }
        },
    },
    { 0 },
};
```

Listing 2: Estructura del servicio GATT custom de SupaClock (`lib/ble_telemetry/ble_telemetry.c`, extracto).

Anexo C: Pipeline ECG con ADC continuo (DMA) y PM lock on-demand

```
#define ECG_DOWNSAMPLE_RATIO 40     /* 20 kHz HW -> 500 Hz */
#define ECG_BLE_CHUNK_SIZE   10     /* 10 x int16 = 20 B/notify */

void ecg_task(void *pvParameter) {
    uint8_t  dma_buf[AD8232_READ_LEN];
    uint32_t ret_num = 0;
    int16_t  ble_chunk[ECG_BLE_CHUNK_SIZE];
    int      chunk_idx = 0;
    uint32_t sum = 0; int count = 0;
    bool     is_dma_running = false;

    while (1) {
        if (!ble_telemetry_is_ecg_mode_active()) {
            if (is_dma_running) {
                ad8232_stop_dma();
                is_dma_running = false;
#if CONFIG_PM_ENABLE
                if (s_ecg_pm_lock) esp_pm_lock_release(s_ecg_pm_lock);
#endif
            }
            vTaskDelay(pdMS_TO_TICKS(500));
            chunk_idx = 0; sum = 0; count = 0;
            continue;
```

```
        } else if (!is_dma_running) {
#if CONFIG_PM_ENABLE
            if (s_ecg_pm_lock) esp_pm_lock_acquire(s_ecg_pm_lock);
#endif
            ad8232_start_dma();
            is_dma_running = true;
        }

        esp_err_t ret = adc_continuous_read(ad8232_get_adc_handle(), dma_buf,
                                            AD8232_READ_LEN, &ret_num,
                                            pdMS_TO_TICKS(100));
        if (ret == ESP_OK) {
            for (int i = 0; i < ret_num; i += sizeof(adc_digi_output_data_t)) {
                adc_digi_output_data_t *p = (adc_digi_output_data_t*)&dma_buf[i];
                uint16_t raw_val = p->type2.data;
                sum += raw_val; count++;
                if (count >= ECG_DOWNSAMPLE_RATIO) {
                    ble_chunk[chunk_idx++] = (int16_t)(sum / ECG_DOWNSAMPLE_RATIO);
                    sum = 0; count = 0;
                    if (chunk_idx >= ECG_BLE_CHUNK_SIZE) {
                        ble_telemetry_send_ecg(ble_chunk, sizeof(ble_chunk));
                        chunk_idx = 0;
                    }
                }
            }
        }
    }
}
```

Listing 3: Tarea de ECG: DMA on-demand → down-sample → BLE notify (`src/tests/test_general.c`, función `ecg_task`, extracto).

Anexo D: Pan-Tompkins en Python+NumPy (validado offline)

```
def detect_r_peaks(integrated: np.ndarray,
                   original:   np.ndarray,
                   fs: float = 100.0) -> list[int]:
    spki = np.max(integrated[:int(2 * fs)]) * 0.25    # Signal peak
    npki = np.mean(integrated[:int(2 * fs)]) * 0.5    # Noise peak
    threshold1 = npki + 0.25 * (spki - npki)

    r_peaks = []
    refractory_period = int(0.2 * fs)                 # 200 ms
    last_peak = -refractory_period

    for i in range(1, len(integrated) - 1):
        if (integrated[i] > integrated[i-1]
                and integrated[i] > integrated[i+1]):
            if integrated[i] > threshold1 and (i - last_peak) > refractory_period:
                spki = 0.125 * integrated[i] + 0.875 * spki
                r_peaks.append(i)
                last_peak = i
            else:
```

```
                npki = 0.125 * integrated[i] + 0.875 * npki
            threshold1 = npki + 0.25 * (spki - npki)
    return r_peaks
```

Listing 4: Detección adaptativa de R-peaks en NumPy (`firebase/functions/main.py`, función `detect_r_peaks`; aún no desplegada como Cloud Function).

Anexo E: Perfiles de energía (`power_profiles[]`)

```
static const power_profile_t s_profiles[POWER_MODE_COUNT] = {
    [POWER_MODE_SPORT] = {
        .hrm_poll_ms          = 100,            /* 10 Hz polling FIFO */
        .hrm_auto_period_ms   = 0,              /* continuo, nunca SHDN */
        .spo2_auto_period_ms  = 5UL * 60 * 1000,/* SpO2 cada 5 min */
        .hrm_shdn_between     = false,
        .imu_poll_ms          = 20,             /* 50 Hz */
        .temp_period_ms       = 30 * 1000,      /* temp cada 30 s */
        .bat_period_ms        = 30 * 1000,      /* bat cada 30 s (igual en
            ↪ todos) */
        .ble_agg_flush_ms     = 1000,           /* flush 1 s */
        .display_off_default_s= 30,
        .name = "SPORT",
    },
    [POWER_MODE_NORMAL] = {
        .hrm_poll_ms          = 100,
        .hrm_auto_period_ms   = 10UL * 60 * 1000, /* HR spot cada 10 min */
        .spo2_auto_period_ms  = 30UL * 60 * 1000, /* SpO2 spot cada 30 min */
        .hrm_shdn_between     = true,
        .imu_poll_ms          = 40,             /* 25 Hz */
        .temp_period_ms       = 5 * 60 * 1000,
        .bat_period_ms        = 30 * 1000,
        .ble_agg_flush_ms     = 10 * 1000,      /* flush cada 10 s */
        .display_off_default_s= 15,
        .name = "NORMAL",
    },
    [POWER_MODE_SAVER] = {
        .hrm_poll_ms          = 100,
        .hrm_auto_period_ms   = 30UL * 60 * 1000, /* HR spot cada 30 min */
        .spo2_auto_period_ms  = 0,              /* sólo manual */
        .hrm_shdn_between     = true,
        .imu_poll_ms          = 80,             /* 12.5 Hz */
        .temp_period_ms       = 15 * 60 * 1000,
        .bat_period_ms        = 30 * 1000,
        .ble_agg_flush_ms     = 60 * 1000,      /* flush cada 60 s */
        .display_off_default_s= 8,
        .name = "SAVER",
    },
};
```

Listing 5: Tabla de perfiles de energía (`lib/power_modes/power_modes.c`, extracto).